

- ① Bit Manipulation
- ② Recursion & Backtracking
- ③ Dynamic Programming
- ④ Greedy Techniques
- ⑤ Graphs, Trees. (Trie)

Bit Manipulation

$\&, |, \text{xor}(\wedge), \gg, \ll$

$$\begin{array}{r} 0101 \\ \& 1100 \\ \hline 0100 \end{array}$$

$\gg$

0010

$2 \gg 3$

$2 \ll 3$

(1) 1101

($\wedge$) 1001

2^5 2^4 2^3 2^2 2^1 2^0
 32 16 (8) 4 2 1

0 0 0 0 1 0
 1 0 0 0

1 0 0 0
 0 $\rightarrow$ 1 0 0
 0 $\rightarrow$ 0 1 0

$8 \gg 2$

*101)
 / / / /

$\sim 0 \rightarrow 1$

$8 \ll 3$

$\begin{array}{ccccccc}
& & & & & & 1 \\
& & & & & & 0 \\
& & & & & 1 & 0 \\
& & & 0 & 1 & 0 & 0 \\
& & 1 & 0 & 0 & 0 & 0 \\
& 1 & 0 & 0 & 0 & 0 & 0
\end{array}$

$4 \ll 1$

$n = 8$

$\frac{n}{2} = 4$

$n = n \gg 1$

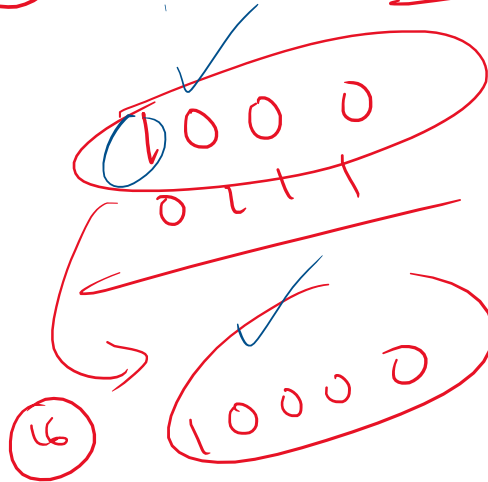
0100

0000 $\rightarrow 8$

1000

0100 $\rightarrow 4$

$\begin{array}{c}
0011 \\
0010
\end{array}$



$n = 1$
 7

$\begin{array}{c}
1111 \\
1110
\end{array}$

$\begin{array}{c}
1100 \\
1011
\end{array}$

	64	32	16	8	4	2	1
64	1	0	0	0	1	0	0
67	1	0	0	0	0	1	1

XOR properties

- ① $a \wedge b = b \wedge a \rightarrow$ Commutative
- ② $(a \wedge b) \wedge c = a \wedge (b \wedge c) \rightarrow$ Associative

① $a \wedge b = b \wedge a \rightarrow$ Commutative

② $a \wedge (b \wedge c) = (a \wedge b) \wedge c \rightarrow$ Associative

③ $(a_1 \& b_1) \wedge (a_2 \& b_2) \wedge (a_2 \& b_1) \wedge (a_2 \& b_2)$

XOR and & Distributive Property.

$(a_1 \wedge a_2) \& (b_1 \wedge b_2)$

$(a_1 \& b_1) \wedge (a_2 \& b_2) \wedge (a_2 \& b_1) \wedge (a_2 \& b_2)$

⑧

→ set bits

⑦

→ 0111 → ③

②

→ 0011 → ② + 1 = ③

10001 → 17 → ②

④

→

$n > 0$

if $(n \& 1) == 1$
 $n = n / 2$

while($n > 0$)
 {
 if $(n \& 1) == 1$
 ++
 }
 }
 $n = n > 1$

MSB LSB
 0011
 0001

 0001

① < K times
 &

Bit Masking:

① < K-1

4 3 2 1
 00101

01000

0001
 0010

01000 → 2nd bit set

K = 3

$$\begin{array}{cccc} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{array}$$

$$\begin{array}{ccc} 0 & 1 & 0 \end{array}$$

$$\underline{\underline{34}}$$

if $(n \& (1 \ll (k-1)) \neq 0)$
 yes
 else no

Given a number N, check if Kth bit is set or Not

oth index

$$\begin{array}{cccc} 3 & 2 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{array}$$

N = 8

K = 4

res = 8 >> K-1 times

$$\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array}$$

$O(\log n)$

n/2

(res & 1 == 1)

 yes
 else No

$O(1)$
 $O(1)$

$$\begin{array}{r} 0001 \\ 0001 \\ \hline 0001 \end{array}$$

Set Bit

Problem: check if Kth bit is set or not in given Number N.

Sample Test Case:

Input:

4 1

Output:

NO

Count number of Set Bits

Problem: count the number of set bits (i.e., bits with value 1) in the binary representation of a given integer.

Input:

5

Output :

2

Explanation:

5 in binary $\rightarrow 0101 \rightarrow 2$ set bits

Brian Kenningham

while (n > 0)

2

~~$n = n \& (n-1)$~~
count++

 ~~$n = n + 1$~~
 ~~$n \& 1 == 1$~~

$\rightarrow 2$
 $\times 100$
 $\hline 010 \mid 4$
 $\hline$
 $\rightarrow 0 \times 00$
 $4 \quad 00 \mid \mid f$
 $\hline$
 0000
 C++
 72

Input:

5

Output:

7

Explanation:

1 → 0001 → 1

2 → 0010 → 1

3 → 0011 → 2

4 → 0100 → 1

5 → 0101 → 2

Total = 1 + 1 + 2 + 1 + 2 = 7 set bits

$O(n \log n)$
 $O(n \times k)$ $\frac{n}{2}$

 $O(n)$ Bit Pattern Analysis

N = 10

	2^3	2^2	2^1	2^0	weight
0 →	0	0	0	0	
1 →	0	0	0	1	
2 →	0	0	1	0	
3 →	0	0	1	1	
4 →	0	1	0	0	
5 →	0	1	0	1	
6 →	0	1	1	0	
7 →	0	1	1	1	
8 →	1	0	0	0	
9 →	1	0	0	1	
10 →	1	0	1	0	

0 1 0 1

for every 2 numbers

0011 0011

4 numbers

0000 1111

Step 1: Calculate pattern length
 Calculate weight of the bit

$$w = 1$$

$$PL = 2$$

$$N = 10 \Rightarrow 10/2 = \underline{5}$$

$$\text{total} = 5 \times PL/2 = 5 \times 2/2 = \underline{5}$$

Step 2:

$$10 \% 4 = \underline{2}$$

if $\text{rem} \geq 2^i$
 $2 \geq 2$ ✓

$$\dots \text{rem} - 2^i + 1$$

$$2 > 2 \checkmark$$

$$total = total + (rem - 2^i) + 1$$

$$\underline{32} \rightarrow \textcircled{5} 100000 = 4 + (2 - 2) + 1 = 4 + 0 + 1 = \underline{5}$$

for (i=0; (1<i) & n; i++)

$$weight = (1 < i)$$

$$pL = (1 < (i+1))$$

$$con = n / pL$$

$$total = con \times pL / 2$$

$$rem = N \% pL$$

if (rem >= (1 < i))

$$total = total + (rem - (1 < i) + 1)$$

Count set bits and store in Array

Problem: Count of Set bits for each number in the range upto N and Store it in array

Input: 5

Output: [0, 1, 1, 2, 1, 2]

dp $dp[n+1]$

dp [0, 1, 1, 2, 1, 2]

0 1 2 3 4 5

$$dp[i] = dp[i/2] + \begin{cases} 1 & \text{if its odd} \\ 0 & \text{if its even} \end{cases}$$

$$= dp[0] + 1 \quad (i \& 1)$$

$$dp[1] \leftarrow = 0 + 1 = 1$$

$$dp[2] = dp[2/2] + 0 = dp[1] + 0 \\ = 1 + 0 = 1$$

$$\underline{\underline{dp[i] = dp[i/2] + (i \& 1)}}$$

Find unique element in the array

Problem: Given an array of size N find the single unique element in the array

Input: 4 1 2 1 2 4 7

Output: 7

$$a \wedge a = 0$$
$$a \wedge 0 = a$$

Find missing number in the given array

Problem: Given an array `nums` containing n distinct numbers in the range $[0, n]$, find the only number that is missing from the given array.

Input: 3

3 0 1

Output: 2

Explanation:

0 is always included in the array. missing number in the range is 2.

$$\begin{aligned} 3 \wedge 0 \wedge 1 &= \textcircled{?}^a \\ 0 \wedge 1 \wedge 2 \wedge 3 &= \textcircled{?}^b \end{aligned} \quad \begin{aligned} \underline{xOR} &= \text{yes} \\ \underline{a \wedge b} &= \end{aligned}$$

X1sY0s to decimal

Problem: Given X number of 1's followed by Y number of 0's find the decimal value

Input: 2 2 (1100)

Output: 12

$O(n)$

$x \rightarrow 1$ } digit
 $y \rightarrow 0$ }
 3 2
 1 1 1 0 0
28
 1100
111 y times
 <<
bit masking

$res = (res \ll x) - 1$
 $res = res \ll y$
 $8 - 1 = 7$
 0111
 $11100 \ll y$

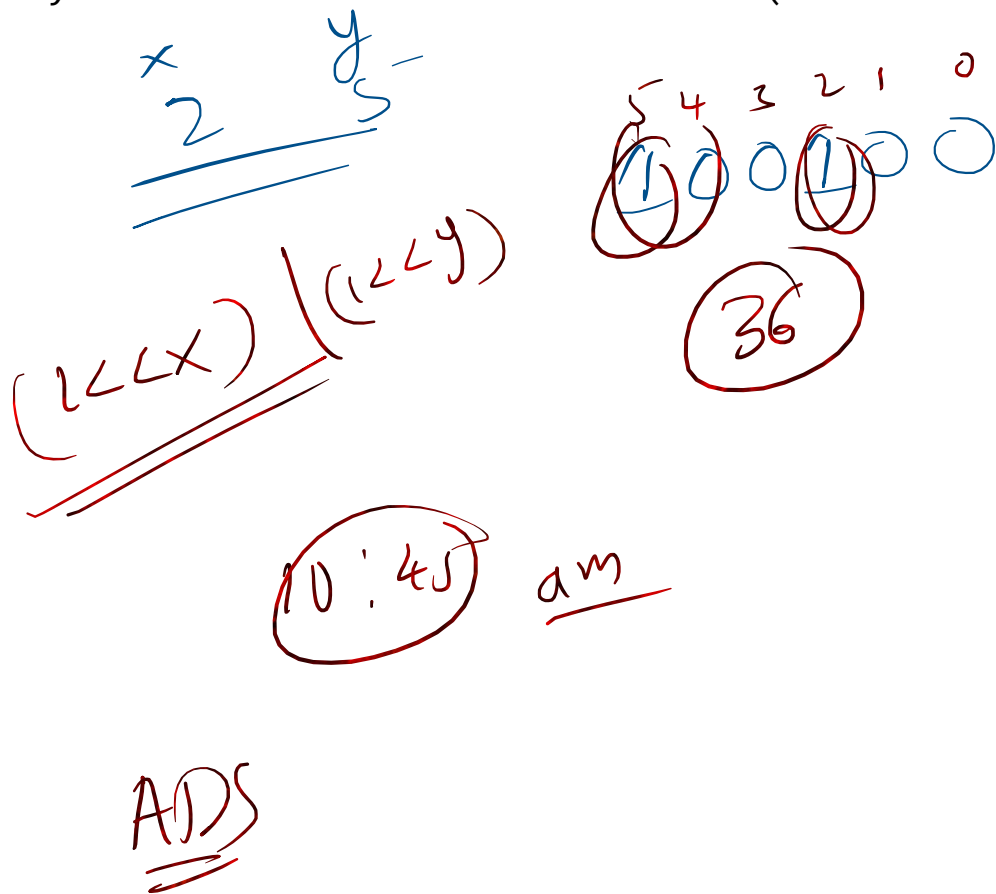
XY set bit to decimal

Problem: You are given two integers, x and y , representing bit positions. Your task is to create a number where **only the x -th and y -th bits are set to 1** (rest all bits are 0).

Input: 2 5

Output: 36

Explanation: binary value will be 100100 which is 36.. (2nd and 5th bit are set)



XoR of sum of Pairs

Problem: Given an integer array `arr` of size `n`, compute the XOR of all pairwise sums of the elements in the array.

Note: Here $(A[i], A[i])$, $(A[i], A[j])$, $(A[j], A[i])$ all are considered as different pairs.

suppose array size is 2 and elements are 1 and 2: $[1, 2]$

i	j	arr[i]	arr[j]	arr[i] + arr[j]
0	0	1	1	2
0	1	1	2	3
1	0	2	1	3
1	1	2	2	4

ans = $0 \wedge 2 \wedge 3 \wedge 3 \wedge 4$
 $= ((2 \wedge 3) \wedge 3) \wedge 4$
 $= (1 \wedge 3) \wedge 4$
 $= 2 \wedge 4$
 $= 6$

Input: `arr[] = {1, 5, 6}`
 Output: 4

$(1, 5, 6)$

$\{ (1,1), (1,5), (1,6), (5,1), (5,5), (5,6), (6,1), (6,5), (6,6) \}$
 $2 \wedge 6 \wedge 7 \wedge 6 \wedge 10 \wedge 11 \wedge 7 \wedge 11 \wedge 12$
 $2 \wedge 10 \wedge 12 = 9 = 4$

for ($i=0; i < n; i++$)

XOR = `XOR ^ arr[i]`

print XOR X 2

Problem: Given an array of n integers, find the sum of xor of all pairs of numbers in the array.

Input : {7, 3, 5}
Output : 12

$7 \wedge 3 = 4$
 $3 \wedge 5 = 6$
 $7 \wedge 5 = 2$
Sum = $4 + 6 + 2$
= 12

$n = \{5, 10, 12, 6, 4\}$
ans = ?
 $5 \rightarrow 0100$

$10 \rightarrow 1010$

$12 \rightarrow 1100$

$6 \rightarrow 0110$

$4 \rightarrow 0100$

$1 \times 4 \times 1 = 4$

$2 \times 3 \times 2 = 12$

$4 \times 1 \times 4 = 16$

$2 \times 3 \times 8 = 48$

80

(10, 6)

(5, 12, 4)

$(5 \wedge 10) + (5 \wedge 6)$

$+ (12 \wedge 10) +$

$(12 \wedge 6) +$

$(4 \wedge 10) + (4 \wedge 6)$

12

$5 \wedge 10 =$
 $5 \wedge 12 =$
 $5 \wedge 6 =$
 $5 \wedge 4 =$
 $10 \wedge 12 =$
 $10 \wedge 6 =$
 $10 \wedge 4 =$
 $12 \wedge 6 =$
 $12 \wedge 4 =$
 $6 \wedge 4 =$

$6 \times 2 = 12$

for (0 to 31)

set = 0
for (i to n)
if (n >> i & 1 == 1)
setbit++

0 (32x7)

unset = n - setbit
total = set x unset x 2ⁱ

$i = 2 \quad 2^2 = 4$

$i = 0 \quad 2^0 = 1$

$i = 1 \quad 2^1 = 2$

{ 2 3 4 5 7 }

$2 \rightarrow 0$
 $3 \rightarrow 0$
 $4 \rightarrow 0$
 $5 \rightarrow 0$
 $7 \rightarrow 0$

$$(3) \times (2) = (6) \times 4 = \underline{\underline{24}}$$

{ 4, 5, 7 }

(2, 3)

for (0 to 3)

32 x 2

(24)

$(2, 4) \neq (2, 5) + (2, 7)$
 $(3, 4) \neq (3, 5) \neq$

$(6) \times 4$
 $= 24$

$\wedge$ $\wedge$ $\&$ $\wedge$

Max Subarray XOR

Problem:

Given an array of integers. The task is to find the maximum subarray XOR value in the given array.

Input: `arr[] = {1, 2, 3, 4}`

Output: 7

Explanation: The subarray {3, 4} has maximum XOR value

Input: `arr[] = {8, 1, 2, 12, 7, 6}`

Output: 15

Explanation: The subarray {1, 2, 12} has maximum XOR value

```
int max = 0
for (int i = 0; i < n; i++)
    int xor = 0
    for (int j = i; j < n; j++)
        xor = xor ^ arr[j]
        if (xor > max)
            max = xor

print max
```

Given an array `arr[]`, we define:

```
prefix[0] = arr[0]
prefix[1] = arr[0] ^ arr[1]
prefix[2] = arr[0] ^ arr[1] ^ arr[2]
...
prefix[j] = arr[0] ^ arr[1] ^ ... ^ arr[j]
```

For array: [1, 2, 3, 4]

Prefix XORs:

```
prefix[0] = 1
prefix[1] = 1 ^ 2 = 3
prefix[2] = 3 ^ 3 = 0
prefix[3] = 0 ^ 4 = 4
```

`arr = [1, 2, 3, 4]`

Assume We want to compute the XOR of subarray from $i = 1$ to $j = 3$, i.e

Normal calculation: $\text{arr}[1] \oplus \text{arr}[2] \oplus \text{arr}[3] = 2 \oplus 3 \oplus 4 = 5$

index prefix[i] (XOR from 0 to i)

```
0    arr[0] = 1
1    arr[0] ^ arr[1] = 1 ^ 2 = 3
2    1 ^ 2 ^ 3 = 3 ^ 3 = 0
3    1 ^ 2 ^ 3 ^ 4 = 0 ^ 4 = 4
```

```
prefix[3] = 4
prefix[i-1] = prefix[0] = 1
```

Step 2: Apply formula

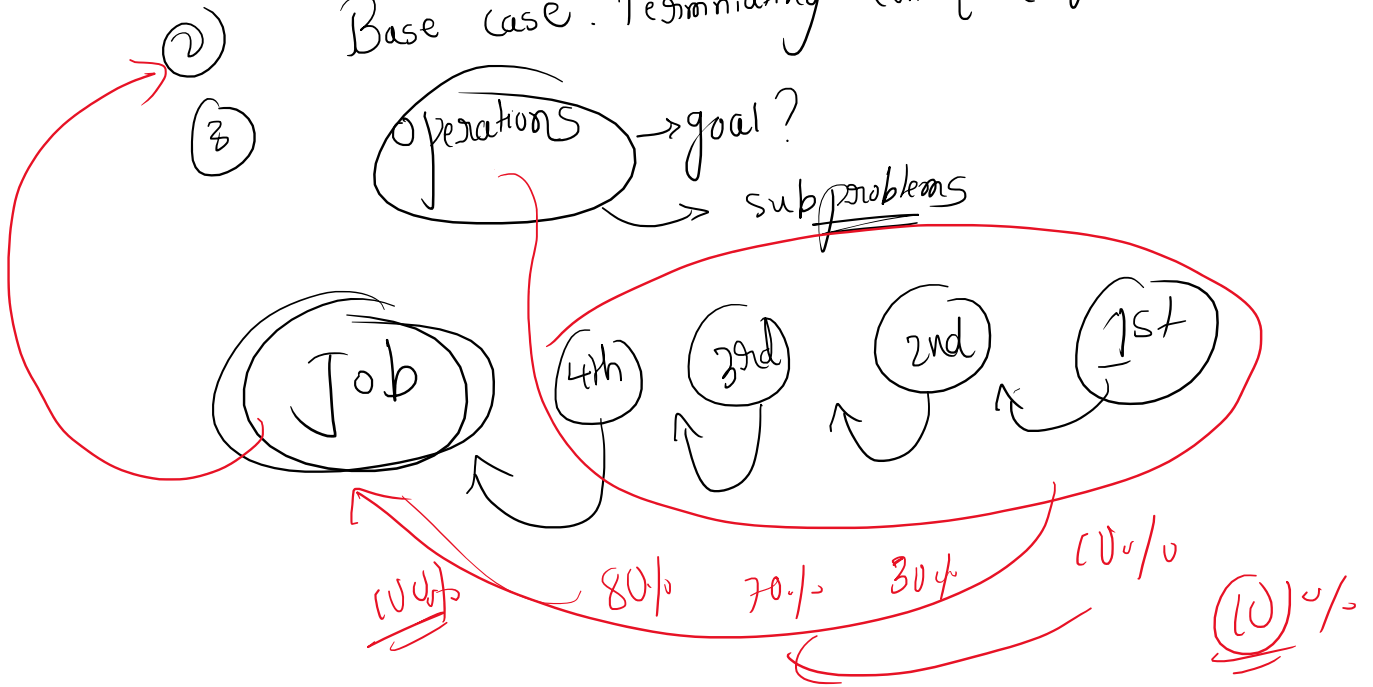
To compute XOR from index $i=1$ to $j=3$:

`subarray_xor = prefix[3] ^ prefix[0] = 4 ^ 1 = 5`

Recursion

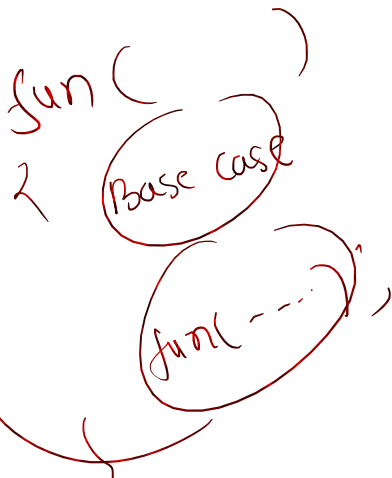
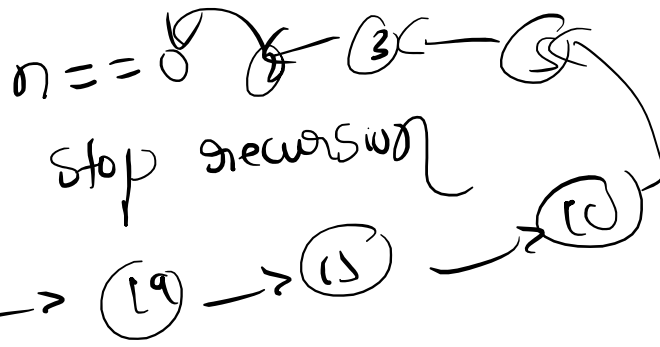
② Function which calls itself ✓

(2) Base case: Terminating criteria (logic)



Recursion should Reach Base Case

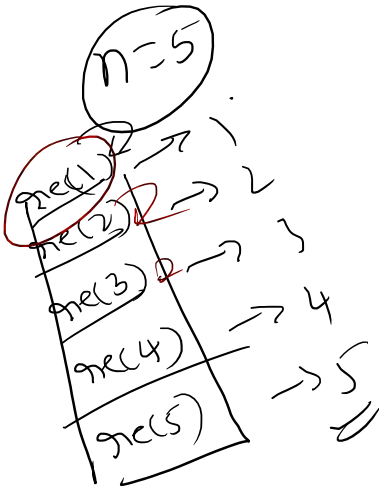
infinite Recursion



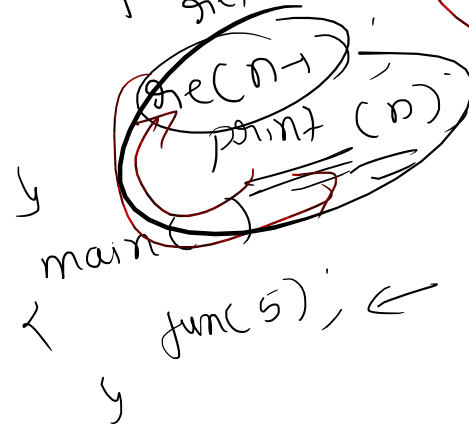
using Recursion



5

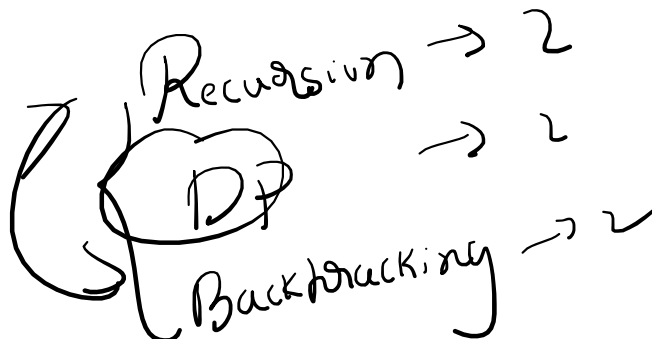
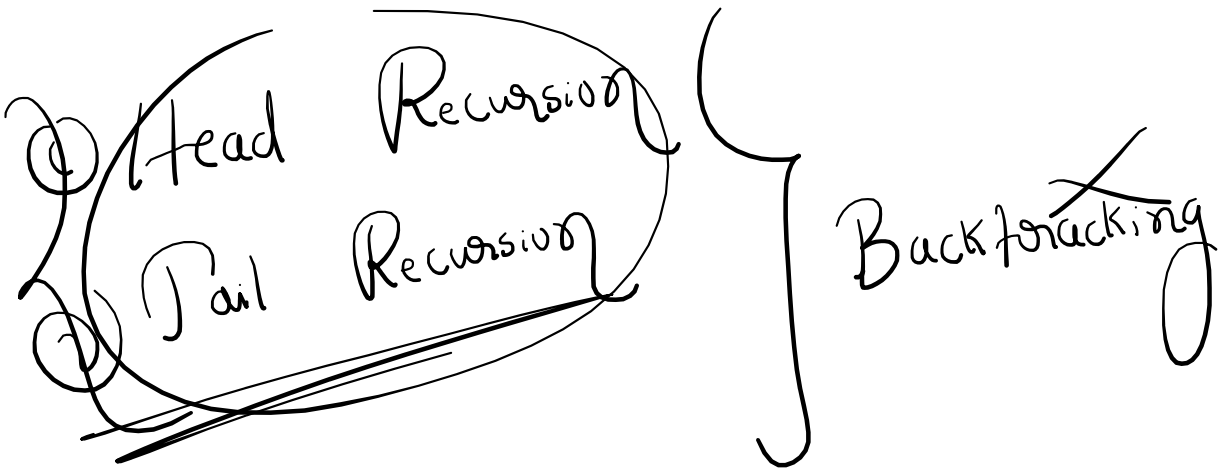


```
func(int n)
{
    if(n==0)
        return x
}
```



1 1 5
2 4
3 2
4 2
5 1

Recursion uses Stack to store calls



M
A

XorSUM of all pairs with AND

Problem:

You are given two integer arrays, arr1 and arr2, both containing non-negative integers. Define a new list containing the bitwise AND of every possible pair formed by taking one element from arr1 and one from arr2

Your task is to find the XOR sum of all elements in this new list, where the XOR sum is the bitwise XOR of all the elements.

Input:

arr1 = [4, 7]

arr2 = [3, 8]

Output:

3

```
int xor_sum = 0
for (int i = 0; i < n; i++)
    for (int j = 0; j < m; j++)
        xor_sum ^= (arr1[i] & arr2[j])
```

Pairs:

4 & 3 = 0

4 & 8 = 0

7 & 3 = 3

7 & 8 = 0

List = [0, 0, 3, 0]

XOR sum = $0 \wedge 0 \wedge 3 \wedge 0 = 3$

→ XOR
→ XOR

4, 7
3, 8
 $(4 \& 3) \wedge (4 \& 8) \wedge (7 \& 3) \wedge (7 \& 8)$
 $(0 \wedge 0 \wedge 3 \wedge 0)$

Bitwise property:

$(a1 \& b1) \wedge (a1 \& b2) \wedge \dots \wedge (a2 \& b1) \wedge (a2 \& b2) \dots$

can be rewritten using distributivity:

$= (a1 \wedge a2 \wedge \dots \wedge an) \& (b1 \wedge b2 \wedge \dots \wedge bm)$

So the final XOR sum is:

$(arr1_xor) \& (arr2_xor)$

Time Complexity: $O(n + m)$

Space Complexity: $O(1)$

- $XOR(arr1) = 4 \wedge 7 = 3$
- $XOR(arr2) = 3 \wedge 8 = 11$ (binary 1011)
- $Result = 3 \& 11 = 3$

Output matches XOR sum = 3

Bitwise XoR of all Pairings

Problem:

You are given two arrays `nums1` and `nums2` containing non-negative integers. Consider all pairs (i, j) where i ranges over indices of `nums1` and j ranges over indices of `nums2`. For each pair, compute the bitwise XOR of `nums1[i]` and `nums2[j]`.
print bitwise xor of all possible pairs from `num1` and `num2`.

Input:

3 4
2 1 3

Output:

7

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < m; j++)  
        result ^= (nums1[i] ^ nums2[j])  
  
System.out.println(result)
```

XOR is commutative and associative:

$a \wedge b = b \wedge a$, and $(a \wedge b) \wedge c = a \wedge (b \wedge c)$

Input:

`nums1` = [2, 1, 3]

`nums2` = [10, 2, 5, 0]

Output= 13

- n = length of the first array (`nums1`)
- m = length of the second array (`nums2`)

[3, 4]

[2, 1, 3]

$O(n)$

(3, 2), (3, 1), (3, 3)

(4, 2), (4, 1), (4, 3)

$$2 \wedge 1 \wedge 3 = \cancel{9 \wedge 2 \wedge 1}$$

$$10 \wedge 2 \wedge 5 \wedge 0 = \text{res 2}$$

$$n = 3$$

$$m = 4$$

Bottom-Up:

- Start with smallest subproblems like $\text{fib}(0)$ and $\text{fib}(1)$
- Build up to $\text{fib}(2)$, then $\text{fib}(3)$, ..., till $\text{fib}(n)$
- You **fill a table** from the bottom

◇ Top-Down:

- Start from $\text{fib}(n)$
- Recursively solve only what's needed
- Then you **recursively break it down into smaller subproblems**, like $\text{fib}(n-1)$, $\text{fib}(n-2)$, etc.
- And **memoize** the smaller ones as you go.
- Use memoization to avoid duplicate computation

fibonacci series recursion $O(2^n)$

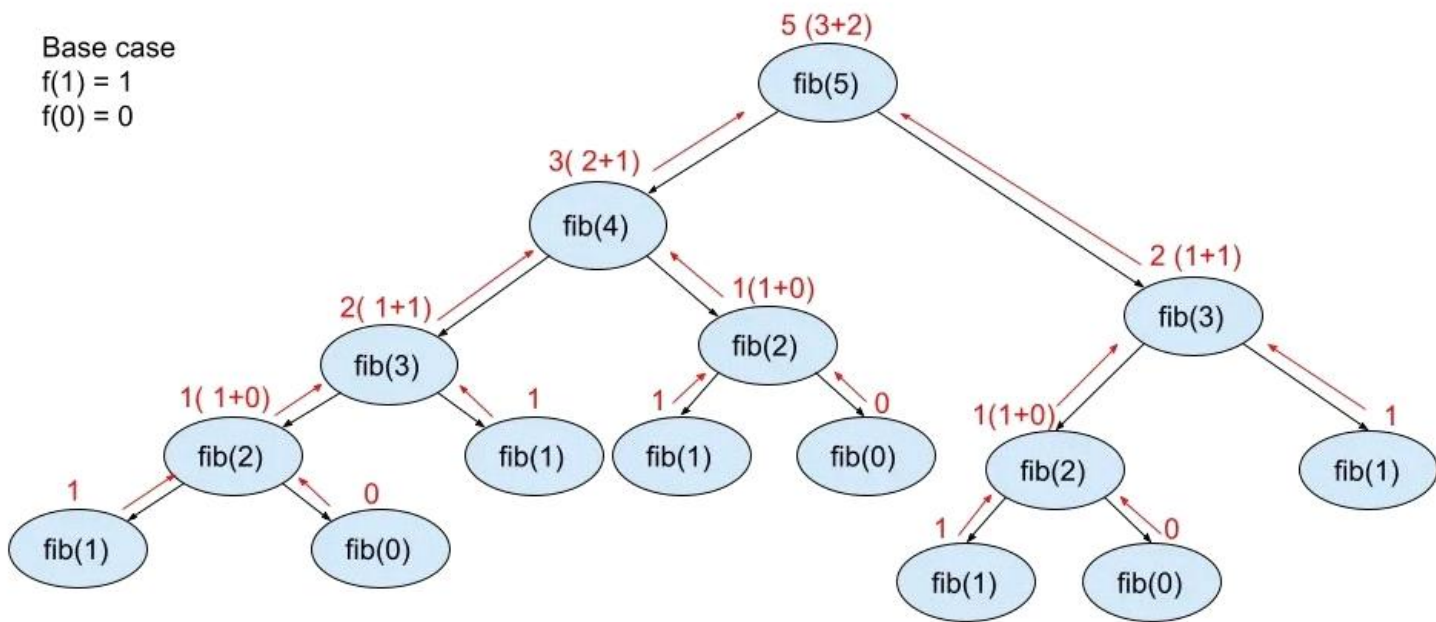
```
int fibonacci(int n) {  
    if (n == 0)  
        return 0;  
    else if (n == 1)  
        return 1;  
    else  
        return fibonacci(n - 1) + fibonacci(n - 2)  
}
```

```
int n = 10  
for (i=0 to n)  
    print(fibonacci(i) + " ")
```

Base case

$f(1) = 1$

$f(0) = 0$



Exponential $O(2^n)$

Approach	Time Complexity	Space Complexity
Naive Recursion	$O(2^n)$	$O(n)$
Recursion + Memoization	$O(n)$	$O(n)$
Iterative DP (Bottom-Up)	$O(n)$	$O(n)$
Iterative with Constant Space	$O(n)$	$O(1)$

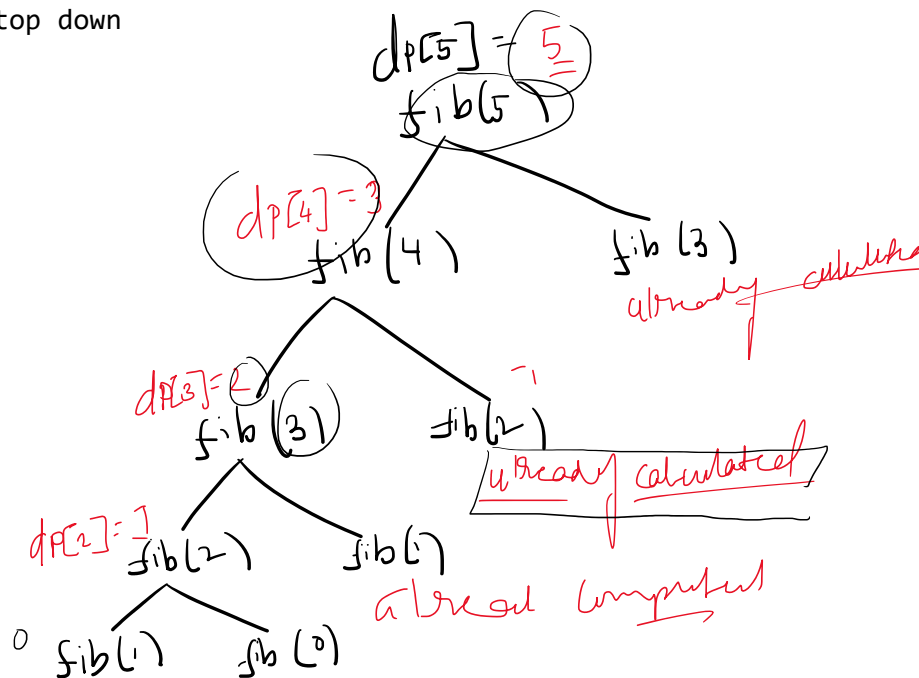
4 $\frac{0111}{0111}$

No Recursion Memoization → No Recursion

Recursion + Backtracking

fibonacci series (top down dp memoization)

24 May 2025 14:49



mbes
3 (5)
(5)
ci
2 (20)
2 (8)

Top-Down DP

- Memoization = storing results of expensive recursive calls so that we don't recompute them.
- It uses a recursive (function call) approach, and stores the result in a dp array or map.

```
int dp = new int[n + 1]
Arrays.fill(dp, -1)
```

```
print(fib(n))
```

```
int fib(int n)
{
    if (n <= 1) return n;
    if (dp[n] != -1)
        return dp[n]
    dp[n] = fib(n - 1) + fib(n - 2)
    return dp[n]
}
```

```
n dp[n]
0 0
1 1
2 1
3 2
4 3
5 5
```

Final dp[] after computation for n = 5:

O(n)

- Each fib(n) is computed **only once**, and stored in dp[].
- Space complexity: O(n) for the array and call stack.

fibonacci series (tabulation bottom up)

24 May 2025 14:50

```
int n = sc.nextInt()
int[] dp = new int[n + 1]

if (n > 0) dp[1] = 1
for (int i = 2; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2]
}
print(dp[n])
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(n)$

fibanccci series space optimized version

24 May 2025 14:52

```
import java.util.*;

int a = 0, b = 1
print(a + " " + b)          // print first two Fibonacci numbers
for (int i = 2; i <= n; i++)
{
    int sum = a + b
    System.out.print(" " + sum) // print next Fibonacci number
    a = b
    b = sum
}
```

Time Complexity:

$O(n)$

Space Complexity:

$O(1)$

Sum of all Subsequences(or subsets same)

05 November 2024 01:28

The total number of subsequences possible is equal to 2^n . (including empty subsequence) elements need not to be contiguous.

Note: (only subsequence and subset can have empty array.)

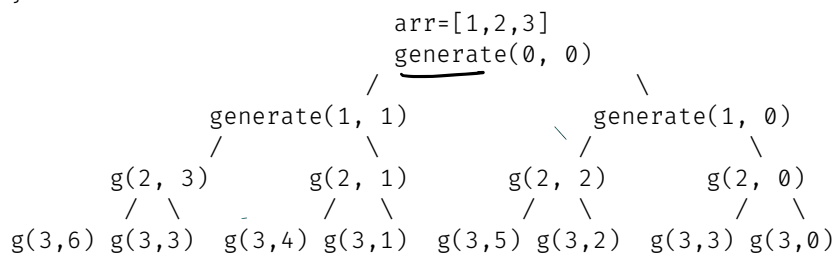
A subsequence is formed by including or excluding each element, maintaining the order.
For array [1, 2]:

- Subsequences = [], [1], [2], [1,2]
- Sum of subsequences = $0 + 1 + 2 + (1+2) = 6$

For an array [1, 2, 3], subsequences are:

[]
[1]
[2]
[3]
[1, 2]
[1, 3]
[2, 3]
[1, 2, 3]

```
void generate(int i, int sum, int[] a, int n)
{
    if(i == n) {
        total += sum;
        return;
    }
    generate(i + 1, sum + a[i], a, n);
    generate(i + 1, sum, a, n);
}
```



Total Sum = $6 + 3 + 4 + 1 + 5 + 2 + 3 + 0 = 24$

ans $\Rightarrow$ [5, 2, 4]

[1, 2, 3] $N=3$

$i=0$ --- (3)
 $i=1$ $i=2$ X

[1, 2, 3]

6 + (10)

$i=0, i=1, i=2$

[1, 3] = 4

In an array subsequence, each element appears in 2^{n-1} subsequences.

first calculate array sum: which is $1+2+3=6$

count of each element: $2^3-1=2^2=4$

print(sum*countofeachelement) $6*4=24$

```

sum = 0
for num in arr
    sum = sum + num

long res = (long) Math.pow(2, n - 1);
or
long res= 1L << (n-1))

print (sum * res)

```

```

import java.util.*;

public class subset_sum_of_all_subsets {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        int total_sum = 0;
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
            total_sum += a[i];
        }
        int power = 1;
        for (int i = 1; i < n; i++) {
            power *= 2;
        }
        int ans = power * total_sum;
        System.out.println(ans);
    }
}

```

The **only difference between subsets and subsequences** is the **order and position of elements**.

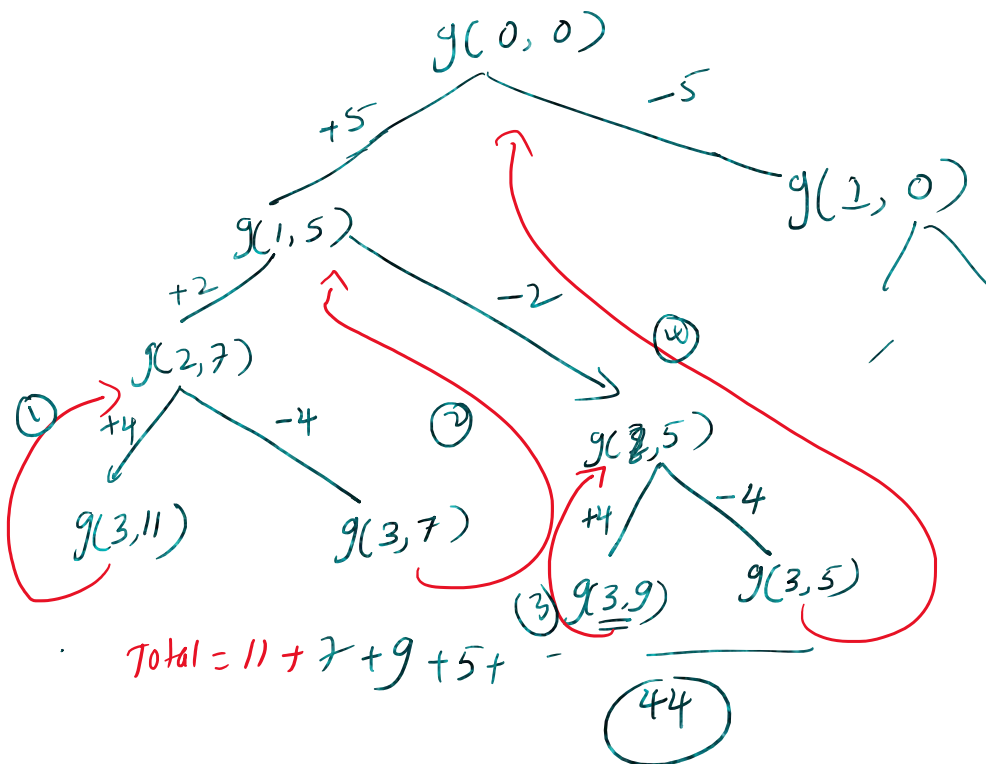
Characteristic	Subset	Subsequence
Order of Elements	The order of elements does not matter. A subset consists of any combination of elements chosen from the original set, regardless of their position. Example: For set $s = \{1, 2, 3\}$, subset $\{3, 2\}$ is equivalent to $\{2, 3\}$.	The elements must appear in the same relative order as in the original sequence, though they do not need to be contiguous . Example: For sequence $s = \{1, 2, 3\}$, subsequence $\{3, 2\}$ is invalid , but $\{1, 3\}$ or $\{2, 3\}$ are valid.
Relationship with Original Set/Sequence	Defined in the context of sets, duplicates are irrelevant.	Defined in the context of sequences, duplicates are preserved.
Size and Selection	Formed by selecting any combination of elements from the set.	Formed by deleting specific elements without rearranging the rest.
Contextual Usage	Primarily used in set theory and combinatorics .	Primarily used in algorithms , string matching , and sequence alignment .

Example

For set $s = \{1, 2, 3\}$, subset $\{3, 2\}$ is valid because order does not matter.

For subsequence $s = \{1, 2, 3\}$, subsequence $\{3, 2\}$ is invalid because order is not preserved.

$[5, 2, 4]$



0/1 knapsack problem using recursion

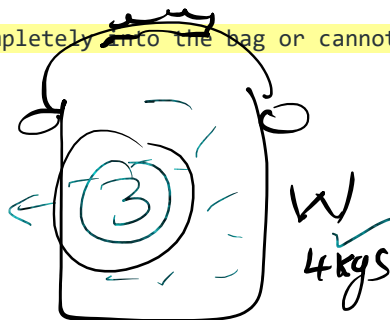
24 May 2025 09:12

→ Recursion

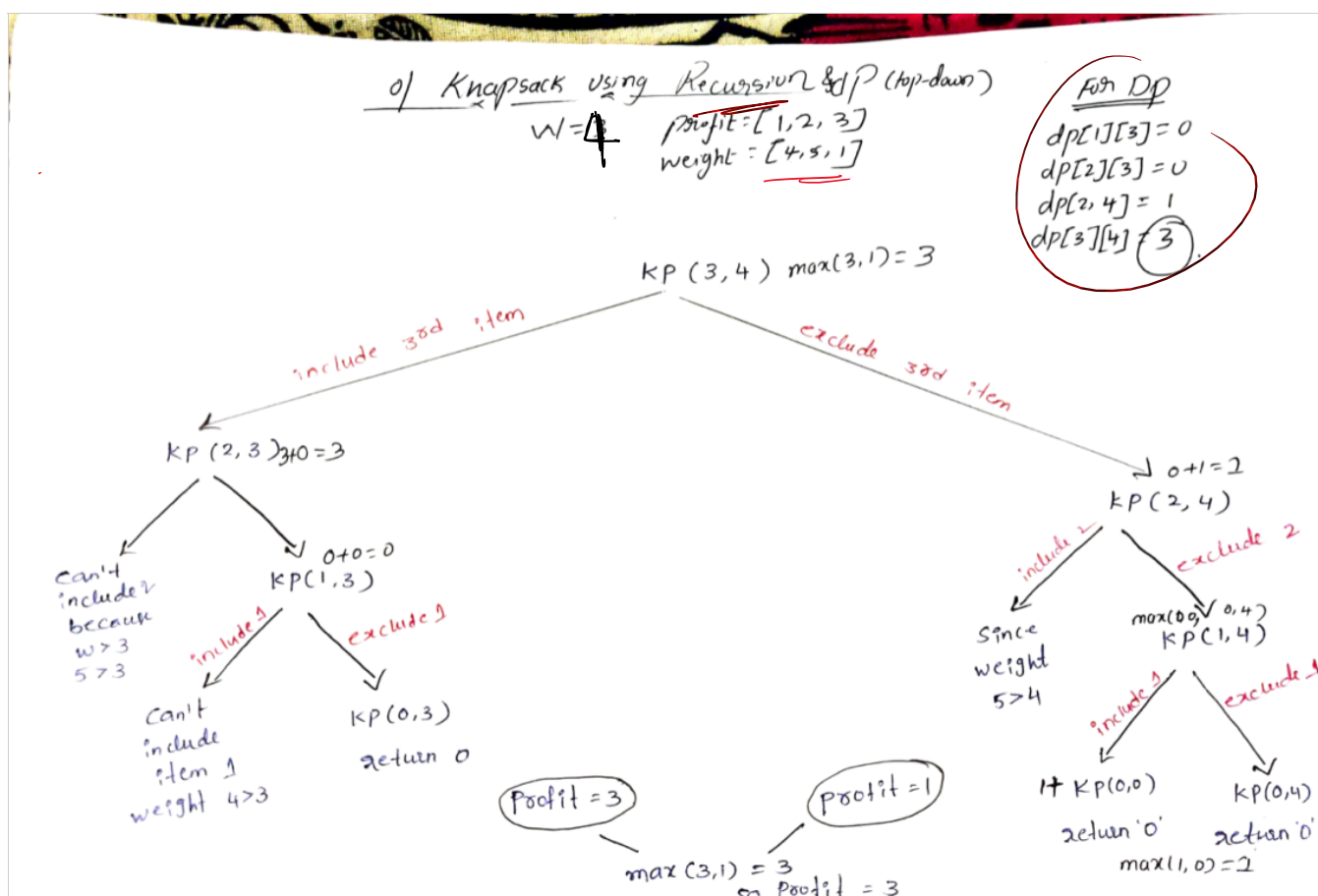
Given n items where each item has some weight and profit associated with it and also given a bag with capacity W , [i.e., the bag can hold at most W weight in it]. The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.

Note: The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Input: $W = 4$, $profit[] = [1, 2, 3]$, $weight[] = [4, 5, 1]$
Output: 3



```
int knapsack(int n, int W, int[] wt, int[] val)
{
    if(n == 0 || W == 0)
        return 0;
    if(wt[n - 1] > W)
        return knapsack(n - 1, W, wt, val);
    else
    {
        int include = val[n - 1] + knapsack(n - 1, W - wt[n - 1], wt, val);
        int exclude = knapsack(n - 1, W, wt, val);
        return Math.max(include, exclude);
    }
}
```



call `knapsack(n-1, W, wt, val, dp)`

```
int knapsack(int n, int W, int[] wt, int[] val, int[][] dp) {
    if(n == 0 || W == 0)
        return 0;
    if(dp[n][W] != -1)
        return dp[n][W]
    if(wt[n - 1] > W)
        dp[n][W] = knapsack(n - 1, W, wt, val, dp)
    else {
        int include = val[n - 1] + knapsack(n - 1, W - wt[n - 1], wt, val, dp)
        int exclude = knapsack(n - 1, W, wt, val, dp)
        dp[n][W] = Math.max(include, exclude)
    }
    return dp[n][W]
}
```


print all subsets backtracking 1

Problem: Print all valid subsets of a given array.

Input: [1,2,3]

Output:

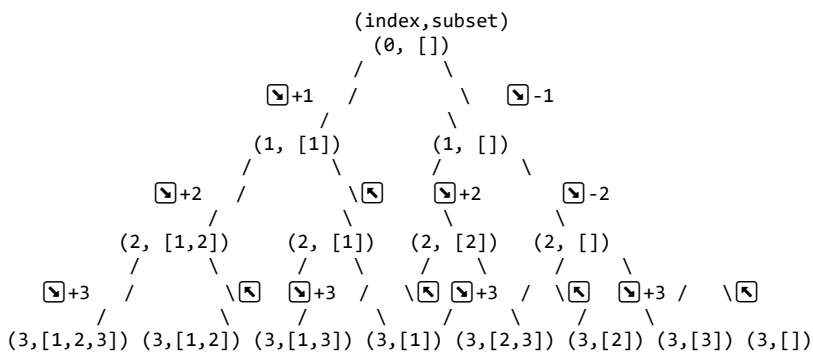
[1, 2, 3]
[1, 2]
[1, 3]
[1]
[2, 3]
[2]
[3]
[]

```
call generateSubsets(nums, 0, subset)

generateSubsets(int[] nums, int index, List<Integer> subset)
{
    if (index == nums.length)
    {
        print(subset)
        return;
    }

    subset.add(nums[index]) // Option 1: Include nums[index]
    generateSubsets(nums, index + 1, subset) // Backtrack: Remove last added element
    subset.remove(subset.size() - 1)
    generateSubsets(nums, index + 1, subset) // Option 2: explore further options
}
```

Backtracking



Print all subsets in the
lexicographical order 2

Problem:

Input:

3

arr = [2, 1, 3]

Output:

[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3],[3]

```
call generateSubsets(nums, 0, subset)

generateSubsets(int[] nums, int index, List<Integer> subset)
{
    if (index == nums.length)
    {
        return;
    }

    subset.add(nums[index]) // Option 1: Include nums[index]

    System.out.println(subset);

    generateSubsets(nums, index + 1, subset) // Backtrack: Remove last added element
    subset.remove(subset.size() - 1)
    generateSubsets(nums, index + 1, subset) // Option 2: explore further options
}
```

Print all valid parentheses recursion

Problem: Given a number n , print all combinations of balanced parentheses of length n .

Note: A sequence of parentheses is balanced if every opening bracket (has a corresponding closing bracket) in the correct order.

Examples:

Input: $n = 2$

Output: "()"

Explanation: Only one valid sequence can be formed using 1 pair of parentheses, and it is balanced.

Input : $n = 4$

Output: "(())", "()()"

Explanation: Two valid balanced sequences are possible with 2 pairs: one with nested parentheses, and one with pairs side by side.

for $n = 2$.

Step 1: Generate All Strings of Length 2 Using '(' and ')'

Total possible combinations = $2^2 = 4$

All strings of length 2:

1. ((
2. ()
3.)(
4.))



there is only one valid path= ()

Step 2: Check for Balance

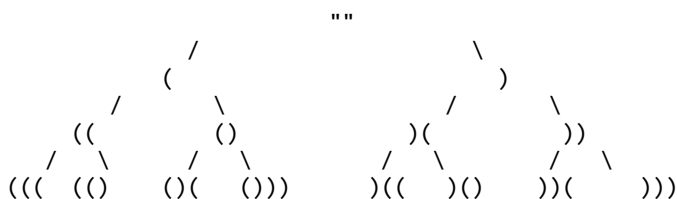
use a **counter method**:

- +1 for '(', -1 for ')'
- If counter < 0 at any point $\rightarrow$ invalid
- At end, if counter $== 0 \rightarrow$ valid

Note: if closing bracket appears before opening its invalid pair as its not balanced paranthesis.

String Steps	Valid
((+1, +1 $\rightarrow$ 2	No
() +1, -1 $\rightarrow$ 0	Yes
)(-1 $\rightarrow$ invalid at start	No
)) -1, -2 $\rightarrow$ invalid	No

for $N=4$



```
generate_all(n, "")
```

```
void generate_all(int n, String s)
{
    if (s.length() == n)
    {
        if (is_valid(s))
        {
            print(s)
        }
        return;
    }

    generate_all(n, s + "(")
    generate_all(n, s + ")")
}

boolean is_valid(String s)
{
    int balance = 0;
    for (int i = 0; i < s.length(); i++)
    {
        if (s.charAt(i) == '(')
            counter++
        else
            counter--

        if (counter < 0)
            return false
    }
    return balance == 0
}
```

Print all valid parantheses backtracking

Problem: Given a number n , print all combinations of balanced parentheses of length n .

Note: A sequence of parentheses is balanced if every opening bracket (has a corresponding closing bracket) in the correct order.

Examples:

Input: $n = 2$

Output: "()"

Explanation: Only one valid sequence can be formed using 1 pair of parentheses, and it is balanced.

Input : $n = 4$

Output: "(())", "() ()"

Explanation: Two valid balanced sequences are possible with 2 pairs: one with nested parentheses, and one with pairs side by side.

- A valid sequence has equal number of '(' and ')'
- At **no point** should closing brackets) exceed opening brackets (
- We **do not place more than $n/2$ opening brackets.**
- Otherwise, we would exceed the valid number of '('.

We build string by adding '(' or ')' at each step under constraints:

- Only add '(' if $\text{open} < n/2$
- Only add ')' if $\text{close} < \text{open}$

If we reach a string of length n and it's balanced $\rightarrow$ **print**

Otherwise, we **backtrack** and try other options.

\\

```
void generate(int n, int open, int close, String s)
{
    if (s.length() == n)
    {
        print(s)
        return
    }
    if (open < n / 2)
        generate(n, open + 1, close, s + "(")

    if (close < open)
        generate(n, open, close + 1, s + ")")
}
```

```
generate_all(n, "");
```

```
void generate_all(int n, String s)
{
    if (s.length() == n)
    {
        if (is_valid(s))
        {
            print(s)
        }
        return;
    }

    generate_all(n, s + "(");
    generate_all(n, s + ")");
}
```

```
boolean is_valid(String s)
{
    int balance = 0;
    for (int i = 0; i < s.length(); i++)
    {
        if (s.charAt(i) == '(')
            counter++
        else
            counter--

        if (counter < 0)
            return false
    }
    return balance == 0
}
```

Psuedo Code Naive Approach

```
if n % 2 != 0
    System.out.println("No valid sequences")
    return
creat empty arraylist result
    generate("", n)
    for String s : result
        System.out.println(s)

static void generate(String current, int n)
    if current.length() == n
        if is_valid(current)
            result.add(current)
        return
    generate(current + "(", n)
    generate(current + ")", n)

static boolean is_valid(String str)
    int count = 0
    for int i = 0; i < str.length(); i = i + 1
        char ch = str.charAt(i)
        if ch == '('
            count = count + 1
        else
            count = count - 1
        if count < 0
            return false
    return count == 0
```

Optimized Approach: using recursion with pruning (only generate valid strings)

- Maintain counts of how many '(' and ')' used so far.
- Only add '(' if count of '(' used < n/2.
- Only add ')' if count of ')' used < count of '(' used (to keep balance).
- This avoids generating invalid sequences altogether, improving efficiency drastically.

Explanation:

- max = number of pairs = n/2.
- open = number of '(' used.
- close = number of ')' used.
- Recursively build string, only adding valid parentheses.

Time Complexity:

- $O(4^n / \sqrt{n})$ – Catalan number complexity (number of balanced parentheses strings).
- Much better than naive $O(n * 2^n)$.

Explanation:

- Each balanced parentheses sequence consists of **pairs** of parentheses.
 - One **pair** is "()" which has length 2 characters.
 - If you want to generate sequences with **n pairs**, total length of the string is always $2 \times n$ characters.
- Why 4?**

- Because there are 2 choices for each of the 2n positions: total combinations = $2^{2n} = 4^n$

The total number of balanced parentheses sequences with **n pairs** is the **nth Catalan number**:

```
ArrayList<String> result = new ArrayList
```

```
    if n % 2 != 0
        print "No valid sequences"
        return
    generate("", 0, 0, n / 2)
    for each String s in result
        print s
```



```

void generate(String current, int open, int close, int max)
    if current.length == max * 2
        result.add(current)
        return
    if open < max
        generate(current + "(", open + 1, close, max)
    if close < open
        generate(current + ")", open, close + 1, max)

```

Why max * 2?

- max = number of pairs (e.g., 2 pairs means max = 2)
- Total length of balanced parentheses string = 2 * max (because each pair has an opening and a closing bracket)

- The "backtracking" is **in the recursion itself** for this problem.
- Because strings are immutable, each recursive call has its own current state.
- After returning from recursion, the caller tries other branches without needing to undo changes explicitly.

[1, 2, 3] k=5

$dp[1] = T$

$dp[i-1]$
 $dp[0]$

	dp_0	(1)	2	(3)	4	(5)
dp		T	T	T	0	T

```

for (i = 1; i < n; i++)
{
    for (sum = k; sum > arr[i]; sum--)
    {
        if (dp[sum - arr[i]])
            dp[sum] = T
    }
    print(dp[k])
}
    
```

$dp[3-2]$

$dp[4-2]$

$dp[2] \times$

$i=1, \text{ sum}=5, 5 \geq 2 \checkmark$ $dp[3]$
 if ($dp[5-2]$)
 {
 }
 x

(2)
 $dp[3-(2)] = dp[1] = T$

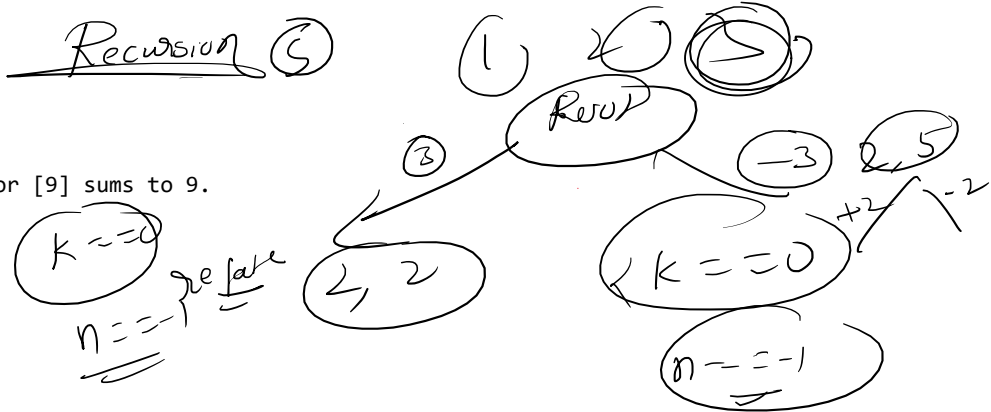
$dp[3] = \text{True}$

Subset with sum K

Problem: Given n integers and an integer k, determine if there exists a subset of the given integers whose sum is exactly k.

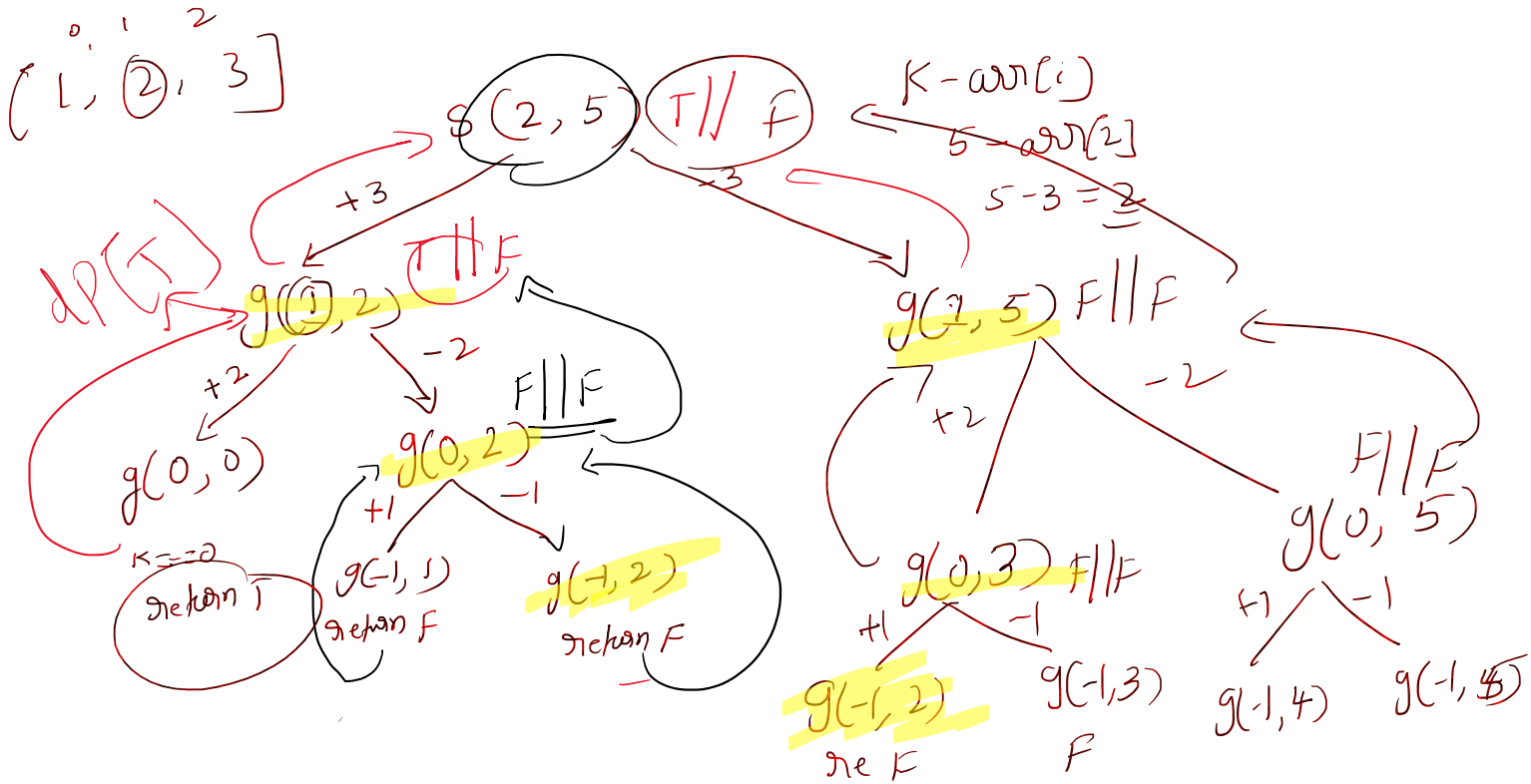
Input:
n = 4
arr = [3, 1, 5, 9]
k = 9
Output: true
Because subset [3, 1, 5] or [9] sums to 9.

n = 3
arr = [1, 2, 3]
k = 5
Output: true



```
public static boolean subset_sum(int[] arr, int n, int k) {
    if(k == 0) return true
    if(n == -1) return false

    return subset_sum(arr, n-1, k-arr[n]) || subset_sum(arr, n-1, k)
}
```



3, 1 → T
3,) → T
2, 1, 1 → T

2, 3, 1, 1, (K=4)

Subsetsum(int i, int k, int lja, int n)

subsetsum(int i, int k, int [] a, (int n))

Call

subsetsum(n-1, k, a, n), {

if (k == 0) return T
if (i == -1) return F

$dp[i][k] = \text{subsetsum}(i-1, k - a[i], a, n)$
 $\text{subsetsum}(i-1, k, a, n)$
return $dp[i][k]$

$dp[0][2] = T$

$dp[0][k]$

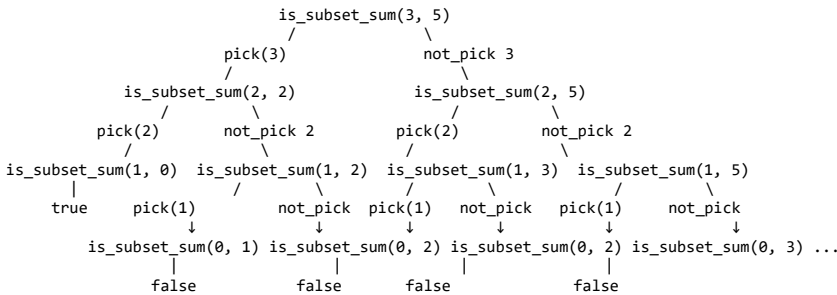
$dp[3][5] = T$

$dp[n][k] \quad s+1$
 $\quad \quad \quad =$

$dp[n][k+1]$

if ($dp[i][k] \neq -1$)
return $dp[i][k]$

recursive decision tree (input= 1 2 3 and sum=5)



subset sum using memoization

- Use a 2D boolean DP array `dp[n][k+1]` to store results of subproblems.
- Avoid recomputation by storing results.

Time Complexity: $O(n \cdot \text{sum})$

Space Complexity: $O(n \cdot k)$ for dp + $O(n)$ recursion stack

Idea:

Store results of already solved subproblems in a DP table:

- Use a 2D array `dp[n][k+1]`
here `k` indicates the sum
the sum `k` can range from 0 to `k` inclusive. hence `k+1`
- `dp[i][k]` means: Is there a subset of `arr[0..i]` that adds up to sum `K`

```
dp = new boolean[n][k + 1]
```

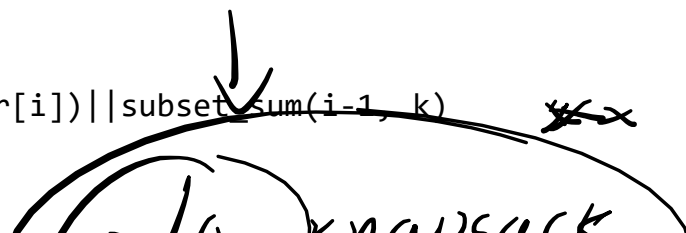
```
call subset_sum(n-1,k)
```

```
public static boolean subset_sum(int i, int k) {
```

```
    if (k == 0) return true;  
    if (i == -1) return false;
```

```
    if (dp[i][k] == true)  
    {  
        return dp[i][k]  
    }
```

```
    return dp[i][k] = subset_sum(i-1, k-arr[i]) || subset_sum(i-1, k)  
}
```



}

for arr=[1,2,3] and sum=5

After full execution, this will be the dp table

i\k	0	1	2	3	4	5
0	1	1	0	0	0	0
1	1	1	1	1	0	0
2	1	1	1	1	1	1

0/1 Knapsack

Tabulation

DP Greedy

for example: $dp[1][2]$:

Can we make a sum of 2 using the elements from index 0 to 1.

it is possible to form sum = 2 using arr[0] and arr[1]

$dp[1][4] = 0 \rightarrow$ Not possible to form 4 using just 1 and 2

input arr = [2, 3, 1, 1], k = 4

In top-down recursion:

- You start with the **last index** $i = n - 1$
- You recurse down to **index 0**
- So valid i values are 0 to $n - 1 \rightarrow$ total n rows
- Therefore, $dp[n][\dots]$ (0-based indexing) is **exact**.

For input $arr = [2, 3, 1, 1]$, $k = 4$, we call:

$subset_sum(3, 4)$ $\leftarrow$ this is the top-level problem

↓

$subset_sum(2, 4)$

↓

$subset_sum(1, 4)$

↓

$subset_sum(0, 4)$

We **start at the top** ($i = n - 1$, $k = \text{sum}$) and go downward into subproblems ($i - 1$, smaller k) – hence called **top-down**.

for counting number of such subsets.


```
dp[n][k] = pick + not_pick;  
return dp[n][k];
```

Subset sum with K space optimized approach

Using Space Optimized DP - $O(n*k)$ Time and $O(k)$ Space

- Instead of a 2D $dp[n][k+1]$ table, we use a 1D boolean array $dp[k+1]$.
- $dp[sum]$ means:
Is there a subset (from elements processed so far) with sum sum .
- We update this array for each element.
- We update sums in **reverse order** (from k down to $arr[i]$) to avoid using the updated values of the same iteration.

$arr = [2, 3, 1, 1]$, $k = 4$

The subset $\{2, 2\}$ isn't valid case.

`boolean[] dp = new boolean[k+1];`

`dp[0] = true`

It represents:

An empty subset can always give sum = 0.

So it's always true from the start.

`if (arr[0] <= k)`

`dp[arr[0]] = true`

`for (int i=1; i<n; i++)`

`{ for (int sum=k; sum>=arr[i]; sum--)`

`{ if(dp[sum-arr[i]])`

`{ dp[sum] = true;`

`}`

`}`

`}`

`}`

`print(dp[k])`

- $dp[sum - arr[i]] == true$ means:

There exists a subset among the elements processed so far whose sum equals $sum - arr[i]$.

- If you add the current element $arr[i]$ to that subset, we can form a subset with sum sum .

Suppose:

- $arr[i] = 3$

- We want to check if a subset sum of $sum = 5$ is possible.

If $dp[5 - 3] = dp[2]$ is true (meaning subset with sum 2 exists), then by adding 3, subset sum 5 can be formed, so $dp[5] = true$.

$arr = [2, 3, 1, 1]$ and $k = 4$

Index (Sum)	0	1	2	3	4
dp (start)	T	F	F	F	F

Final DP Table

Sum	0	1	2	3	4
dp	T	T	T	T	T

Meaning:

You can form all sums from 0 to 4 using subsets of {2, 3, 1, 1}.

Some subsets:

- Sum 1 $\rightarrow$ {1}
- Sum 2 $\rightarrow$ {2}
- Sum 3 $\rightarrow$ {2,1} or {3}
- Sum 4 $\rightarrow$ {3,1} or {2,1,1}

0/1 knapsack problem using recursion

24 May 2025 09:12

Given n items where each item has some weight and profit associated with it and also given a bag with capacity W , [i.e., the bag can hold at most W weight in it]. The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.

Note: The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Input: $W = 4$, $\text{profit}[] = [1, 2, 3]$, $\text{weight}[] = [4, 5, 1]$

Output: 3

Explanation: There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.

```
int knapsack(int n, int W, int[] wt, int[] val)
{
    if(n == 0 || W == 0)
        return 0;
    if(wt[n - 1] > W)
        return knapsack(n - 1, W, wt, val)
    else
    {
        int include = val[n - 1] + knapsack(n - 1, W - wt[n - 1], wt, val)
        int exclude = knapsack(n - 1, W, wt, val)
        return Math.max(include, exclude)
    }
}
```


0/1 knapsack problem using DP memoization top down

24 May 2025 11:19

Both recursive approach and DP versions have the **same recursive structure**. **BUT**, the DP version **remembers the answer** for (n, W) and reuses it.

Approach

dp[i][w] = max value using items 0...i with remaining capacity w

function:

```
knapsack(int n, int W, int[] wt, int[] val, int[][] dp)
```

```
    if(n == 0 || W == 0)
```

```
        return 0
```

```
    if(dp[n][W] != -1)
```

```
        return dp[n][W]
```

```
    if(wt[n - 1] > W)
```

```
        dp[n][W] = knapsack(n - 1, W, wt, val, dp)
```

```
    else {
```

```
        int include = val[n - 1] + knapsack(n - 1, W - wt[n - 1], wt, val, dp)
```

```
        int exclude = knapsack(n - 1, W, wt, val, dp)
```

```
        dp[n][W] = Math.max(include, exclude)
```

```
    }
```

```
    return dp[n][W]
```

knapsack using Tabulation bottom up DP

24 May 2025 13:13

Bottom-Up DP (Tabulation) – Key Idea

We use a $dp[n+1][W+1]$ table where:

- $dp[i][j]$ = maximum profit using first i items and capacity j

For every item i and capacity j :

- If the item's weight is more than capacity j , we can't pick it
 $dp[i][j] = dp[i-1][j]$

- Else, we have 2 choices:

- Include it: $profit = profit[i-1] + dp[i-1][j - weight[i-1]]$
- Exclude it: $profit = dp[i-1][j]$

Take maximum:

$$dp[i][j] = \max(dp[i-1][j], profit[i-1] + dp[i-1][j - weight[i-1]])$$

If you include the i -th item, then:

- You gain its profit: $profit[i - 1]$ // bcz 0 index
- But you reduce available capacity by $weight[i - 1]$
- So now, you can only get additional profit from the remaining capacity

So, the total profit if you include this item is:

$$profit[i - 1] + profit_from_remaining_capacity$$

Which is:

$$profit[i - 1] + dp[i - 1][j - weight[i - 1]]$$

Initialization

- If $i == 0$ or $j == 0$, then $dp[i][j] = 0$
(0 items or 0 capacity $\Rightarrow$ 0 profit)
- The value at $dp[n][W]$ contains the maximum profit

$W = 4$

$profit = [1, 2, 3]$

$weight = [4, 5, 1]$

$O(n * W) \rightarrow$ loop over all items and capacities

$O(n * W)$ for full DP table space

0/1 Knapsack problem using Tabulation (Bottom-up Approach)

$n=4, W=8, weights = \{3, 4, 5, 6\}, profits = \{2, 3, 4, 1\}$

$[0, 2]$ 3, 5, 10, 8

weight[3]=2
↑
previous Row
↑
8-5=3

P_i	W_i	$i \backslash W$	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0	0	0
2	3	1	0	0	0	2	2	2	2	2	2
3	4	2	0	0	0	2	3	3	5	5	5

0/1 Knapsack problem using Tabulation (Bottom-up Approach)

$n=4$, $W=8$, weights = { 3, 4, 5, 6 }
profits = { 2, 3, 4, 1 }

3, 5, 10, 8
[0, 2]

DP table

P_i	w_i	$i \backslash w$	0	1	2	3	4	5	6	7	8
2	3	0	0	0	0	0	0	0	0	0	0
3	4	1	0	0	0	2	2	2	2	2	2
4	5	2	0	0	0	2	3	4	4	5	5
1	6	3	0	0	0	2	3	4	4	5	6

weight[3]=2
↑
previous Row
↑
8-5=3
↑
max(4+2, 5)
=6
4 is current profit
2 is additional profit from remaining capacity

Final profit

procedure: Compare w_i with column weight for example in step 1

$3 > 0$ so we can not include item 1 so profit remains 0.

$3 > 2 \rightarrow 0$, $3 > 3$ include profit of 1st item & fill (2)

step 2: $4 > 1 \rightarrow 0$, $4 > 2 \rightarrow 0$, $4 > 3$ include previous profit which is 2.

$4 > 4$ X now include current profit 3, then $4-4=0$ select $dp[i-1][0] \rightarrow 0$.
Select $\max(3+0, 2) = 3$

(1=1)

1, 2, 3
w 4, 5, 1
0, 2

$W=4$

```
int[][] dp = new int[n + 1][W + 1]
```

```
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= W; j++) {
        if(weight[i - 1] <= j) {
            int include = profit[i - 1] + dp[i - 1][j - weight[i - 1]];
            int exclude = dp[i - 1][j];
            dp[i][j] = Math.max(include, exclude);
        } else {
            dp[i][j] = dp[i - 1][j];
        }
    }
}
```

To print which items are selected

```
List<Integer> selected = new ArrayList<>()
int i = n;
int j = W;
```



```

while (i > 0 && j > 0) {
    if (dp[i][j] != dp[i - 1][j]) {
        selected.add(i - 1)
        j -= weight[i - 1]
    }
    i--
}

```

		0	1	2	3	4
P	W	0	0	0	0	0
	1	1	0	0	0	1
	2	2	0	0	0	1
	3	3	0	3	3	3
	4					

Fractional Knapsack Problem

26 May 2025 15:30

Greedy Approach (Optimal for fractional)

Idea:

- Compute value per weight
- Sort items by value per weight descending
- Pick full items till capacity is left
- Pick fraction of next item if needed

val[] = [60, 100, 120]

wt[] = [10, 20, 30]

capacity = 50

- Sort items by profit/weight ratio ↓: [Item 1, Item 2, Item 3] (descending)

$\sqrt{w}$ Ratio

↓
6 5 3 5

Fractional Knapsack
Greedy Approach

$O(n \log n)$
Since sorting by P/w ratio.

Total weight = 50
profit = [60, 100, 120]
weights = [10, 20, 30]

Step 1: Calculate profit/ratio for each item.

P → 60, 100, 120
W → 10, 20, 30
6 5 4

Step 2: Sort elements based on P/w ratio (highest to lowest)

Step 3:

Step	Action	Capacity left	value	Total
1	pick item 1 (60)	50-10=40	+60	60
2	pick item 2 (100)	40-20=20	+100	160
3	Since 3rd item weight 30 > 20 take 20/30	20/30=0.66	0.66x120=80	160+80=240

50 10

$$50 - 10 = 40$$

$$30 > 20 \times$$

$$20/30 = 0.66 \times 120$$

```
for (int i = 0; i < n; i++) {  
    if (capacity >= items[i].weight) {  
        capacity -= items[i].weight;  
        result += items[i].value;  
    } else {  
        result += items[i].value * (capacity / items[i].weight);  
        break;  
    }  
}
```

24 May 2025 14:24

You can use each coin unlimited times.

You want to give change using the fewest coins possible.

Coins available: ~~25~~, ~~10~~, ~~5~~, ~~1~~.

Visualization for target amount 63:

Take 1 $\rightarrow$ Remaining: 0

Answer: 6 coins.

Doesn't always give optimal for all coin systems (e.g., coins: 9, 6, 1 and amount 12 $\rightarrow$ greedy gives $9+1+1+1 = 12$ coins but optimal is $6+6 = 2$ coins).

- Time: $O(n \log n)$ (highest to lowest denominations)
 - $O(n \log n + \text{amount}/\text{mincoins})$ for sorting
 - Worst-case inner loop: if using all 1s $\rightarrow O(\text{amount})$
- Space: $O(\text{amount})$ for storing result coins (worst-case)

amount = 93

50 20 20 2 1

Approach

1. Sort the coins in descending order.
2. Initialize:
 - remaining_amount = total_amount
 - coin_list = [] (to store selected coins)
3. Loop through each coin c in the sorted list:
 - While remaining_amount >= c, pick the coin:
 - remaining_amount -= c
 - Add c to coin_list
4. Repeat until the amount becomes 0.

```
int amount = sc.nextInt()
```

```
Arrays.sort(coins)
int count=0;
for (int i = n - 1; i >= 0; i--) {
    while (amount >= coins[i]) {
        amount = amount - coins[i];
        count++;
    }
}
print count
```

String Partitioning

Given a string `s` and a dictionary `wordDict`, determine if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Input: `s = "applepenapple", wordDict = ["apple","pen"]`

Output: `true`

Explanation: Return true because "applepenapple" can be segmented as "apple pen apple". Note that you are allowed to reuse a dictionary word.

Example 3:

Input: `s = "catsanddog", wordDict = ["cats","dog","sand","and","cat"]`

Output: `false`

Recursion without memoization

At every position `i` in the string `s`, try **all possible prefixes** `s[0...j]` and:

- If prefix is in dictionary, recursively check the **remaining suffix** `s[j+1:]`.

Input:

```
s = "applepenapple"
wordDict = ["apple", "pen"]
```

Check if "applepenapple" can be segmented using words from the dictionary ["apple", "pen"].

Dictionary:

```
dict = {"apple", "pen"}
```

We will recursively try all **prefixes** of the current string and proceed only when the prefix is in the dictionary.

Step 1: Call with `s = "applepenapple"`

Check all prefixes:

```
"a"
"ap"
"app"
"appl"
"apple" → in dictionary → Recurse on "penapple"
```

Step 2: Call with `s = "penapple"`

Check all prefixes:

- "p"
"pe"
"pen" → in dictionary → Recurse on "apple"

Step 3: Call with `s = "apple"`

Check all prefixes:

```
"a"
"ap"
"app"
"appl"
"apple" → in dictionary → Recurse on ""
```

Step 4: Call with `s = ""` → Reached empty string → return true

```
Step 3 returns true
Step 2 returns true
Step 1 returns true
```

FINAL RESULT: true

"apple" + "pen" + "apple"

```

Set<String> dict = new HashSet<>()
for (int i = 0; i < n; i++)
    dict.add(sc.nextLine())
}
boolean res = word_break(s, dict)
print(res)

boolean word_break(String s, Set<String> dict)
{
    if (s.length() == 0)
        return true
    for (int i = 1; i <= s.length(); i++)
        String prefix = s.substring(0, i)
        if (dict.contains(prefix))
            if (word_break(s.substring(i), dict))
                return true

    return false
}

```

}

0 1 2 3 4 5 6 7 8 9 10 11 12
apple pen apple

using DP ($O(n^2)$)

We use a $dp[i]$ array where:

- $dp[i] = \text{true}$ means $s[0..i-1]$ can be segmented using words from the dictionary.

```

boolean[] dp = new boolean[len + 1]
dp[0] = true
for (int i = 1; i <= len; i++) {
    for (int j = 0; j < i; j++) {
        if (dp[j] && dict.contains(s.substring(j, i))) {
            dp[i] = true
            break
        }
    }
}

```

$sub(0, 5)$

$dp[5] = \text{true}$

$sub(0, 6)$

$dp[6] = \text{true}$

$dp[12] = \text{true}$

$dp[1] = \text{true}$

```
s = "applepenapple"
wordDict = ["apple", "pen"]
```

```
s.length() = 13
```

```
dp array size = 14 (0 to 13)
```

```
dp[0] = true (empty string can be segmented)
```

```
dp[i] means s[0..i-1] can be segmented into dictionary words.
```

```
dp = [T, F, F, F, F, F, F, F, F, F, F, F, F, F]
Index: 0  1  2  3  4  5  6  7  8  9 10 11 12 13
```

```
Iteration over i from 1 to 13:
```

```
i = 1
```

```
Check j = 0: s[0..0] = "a" not in dict → dp[1] = false
```

```
i = 2
```

```
j = 0: "ap" no
```

```
j = 1: "p" no → dp[2] = false
```

```
i = 3
```

```
j=0: "app" no
```

```
j=1: "pp" no
```

```
j=2: "p" no → dp[3] = false
```

```
i = 4
```

```
j=0: "appl" no
```

```
j=1: "ppl" no
```

```
j=2: "pl" no
```

```
j=3: "l" no → dp[4] = false
```

```
i = 5
```

```
j=0: "apple" → YES, in dict and dp[0] = true
```

```
→ dp[5] = true, break inner loop
```

```
dp = [T, F, F, F, F, T, F, F, F, F, F, F, F, F]
```

```
i = 6
```

```
j=0: "applep" no
```

```
j=1: "pplep" no
```

```
j=2: "plep" no
```

```
j=3: "lep" no
```

```
j=4: "ep" no
```

```
j=5: "p" no → dp[6] = false
```

```
i = 7
```

```
similarly check substrings ending at 7 → no match
```

```
dp[7] = false
```

```
i = 8
```

```
j=0 to 4 no matches
```

```
j=5: s[5..7] = "pen" YES in dict, dp[5] = true
```

```
→ dp[8] = true, break inner loop
```

```
dp = [T, F, F, F, F, T, F, T, F, F, F, F, F, F]
```

```
i = 9
```


check substrings ending at 9 → no matches

dp[9] = false

i = 10

no matches

dp[10] = false

i = 11

no matches

dp[11] = false

i = 12

no matches

dp[12] = false

i = 13

j=0 to 7 no matches

j=8: s[8..12] = "apple" in dict, dp[8] = true → YES

→ dp[13] = true

dp = [T, F, F, F, F, T, F, F, T, F, F, F, F, T]

Final dp array:

r

Index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13

dp[] = [T, F, F, F, F, T, F, F, T, F, F, F, F, T]

dp[13] = true means the whole string "applepenapple" can be segmented as dictionary words.

"apple" at indices [0..4]

"pen" at indices [5..7]

"apple" at indices [8..12]

Number of ways to Partition String

Given a string s and a set of dictionary words $dict$, Count how many ways the string can be broken into valid dictionary words.

Note: all segmentations must start from the beginning of the string s .

So it is mandatory to begin matching from index 0 of the string and proceed left to right

Input:

$s = \text{"applepenapple"}$

$dict = [\text{"apple"}, \text{"pen"}]$

Output:

1

Because only one valid way: "apple pen apple"

Input:

$s = \text{"pineapplepenapple"}$

$dict = [\text{"apple"}, \text{"pen"}, \text{"applepen"}, \text{"pine"}, \text{"pineapple"}]$

Output:

3

Explanation:

The string "pineapplepenapple" can be segmented in 3 ways:

"pine" + "apple" + "pen" + "apple" (1)

"pineapple" + "pen" + "apple" (2)

"pine" + "applepen" + "apple" (3)

```
int len = s.length()
int[] dp = new int[len + 1]
dp[0] = 1
```

```
for (int i = 1; i <= len; i++)
    for (int j = 0; j < i; j++)
        String sub = s.substring(j, i)
        if (dict.contains(sub)) {
            dp[i] += dp[j]
```

$s = \text{"pineapplepenapple"}$
Length = 17
Initialize:

$dp[0] = 1$ (empty string)

$dp[1]$ to $dp[17] = 0$

Now iterate:

At $i = 4$:

- $s[0..4] = \text{"pine"}$ is in dict
→ $dp[4] += dp[0] = 1$
→ $dp[4] = 1$

At $i = 9$:

- $s[4..9] = \text{"apple"}$ is in dict
→ $dp[9] += dp[4] = 1$
→ $dp[9] = 1$
- $s[0..9] = \text{"pineapple"}$ is in dict
→ $dp[9] += dp[0] = 1$
→ $dp[9] = 2$

At $i = 12$:

- $s[9..12] = \text{"pen"}$ is in dict
→ $dp[12] += dp[9] = 2$
→ $dp[12] = 2$

At $i = 12$:

- $s[4..12] = \text{"applepen"}$ is in dict
→ $dp[12] += dp[4] = 1$
→ $dp[12] = 3$

At $i = 17$:

- $s[12..17] = \text{"apple"}$ is in dict

Pineapplepenapple

Number of ways to partition

→ $dp[17] += dp[12] = 3$
→ $dp[17] = 3$

Final DP Table

Index (i) dp[i] s[0..i-1]

0	1	""
4	1	"pine"
9	2	"pineapple"
12	3	"pineapplepen"
17	3	"pineapplepenapple"

→ So, $dp[17] = 3 = \text{number of ways to partition.}$

```
Function String_partitioning(s: string, dict: set of words):  
    n = length of s  
  
    dp = array of boolean of size (n + 1)  
    dp[0] = true    // base case  
  
    for i from 1 to n:  
        for j from 0 to i:  
            if dp[j] == 1 AND substring s[j to i-1] in dict  
                dp[i] = true  
                break  
    return dp[n]
```

Print all valid String partitions Backtracking

Print all valid partitions of a string `s` using a given dictionary

Given:

- String: `s`
- Dictionary: `wordDict`

Task: Print all ways to split `s` into dictionary words such that the entire string is used and every substring in the partition exists in the dictionary.

Input

pineapplepenapple

5

apple

pen

applepen

pine

pineapple

Output

pine apple pen apple

pineapple pen apple

pine applepen apple

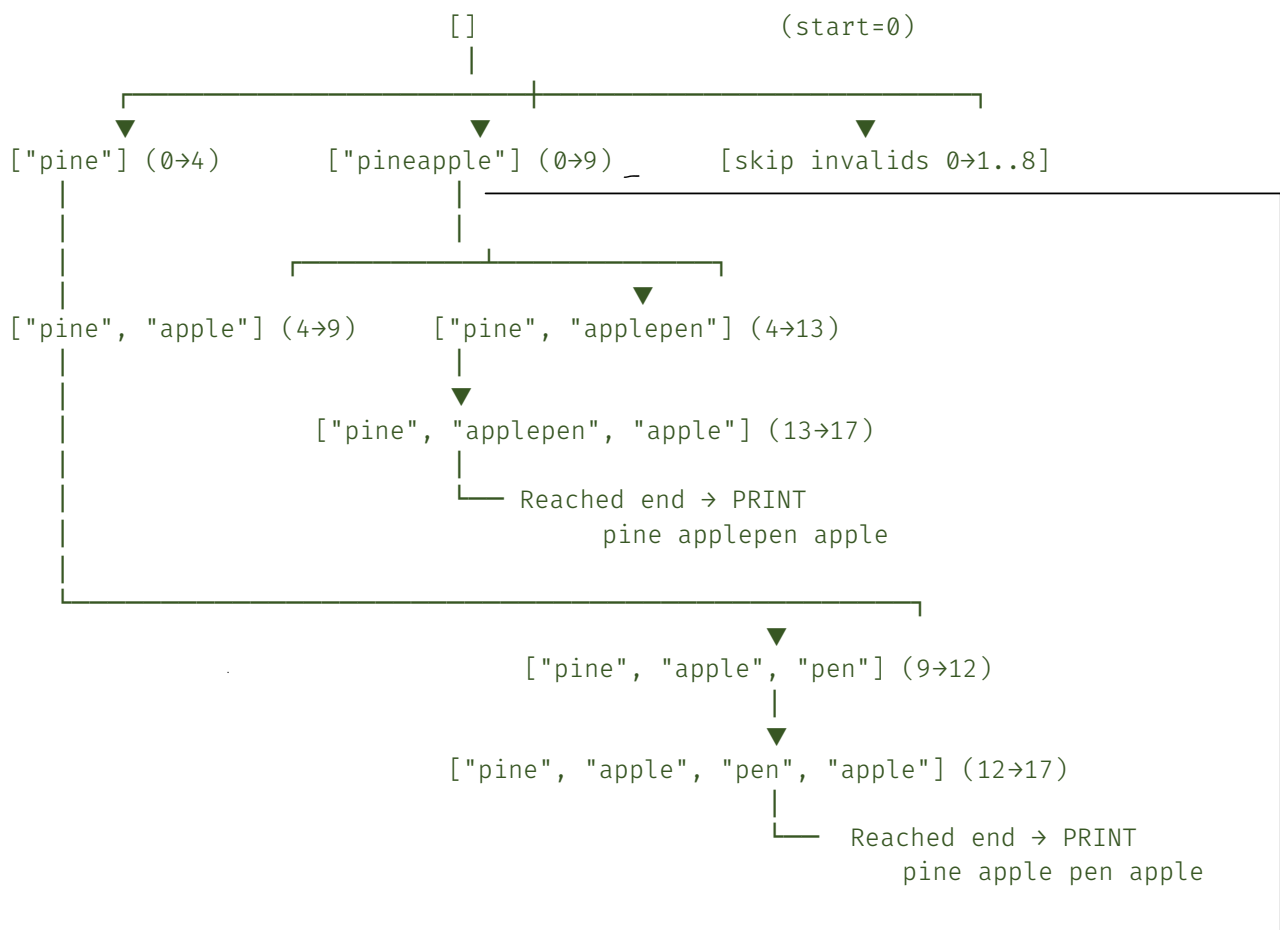
```
s = "pineapplepenapple"
```

```
dict = ["apple", "pen", "applepen", "pine", "pineapple"]
```

We are calling:

```
backtrack("pineapplepenapple", 0, dict, [])
```

We'll build the string from `start = 0`, and at each step we check every substring `s[start:end]` and recurse when it's a valid dictionary word.



Reached end → PRINT
pine apple pen apple

["pineapple", "pen"] (9→12)

["pineapple", "pen", "apple"] (12→17)

Reached end → PRINT
pineapple pen apple

```
void backtrack(String s, int start, HashSet<String> dict, List<String> path) {  
    if (start == s.length()) {  
        System.out.println(String.join(" ", path));  
        return;  
    }  
    for (int end = start + 1; end <= s.length(); end++)  
    {  
        String sub = s.substring(start, end);  
        if (dict.contains(sub)) {  
            path.add(sub);  
            backtrack(s, end, dict, path);  
            path.remove(path.size() - 1);  
        }  
    }  
}
```

Sum of all Subarrays

05 November 2024 01:28

The total number of subarrays in an array of size N is $N * (N + 1) / 2$. elements should be contiguous.

(empty subarray is not allowed)

For an array $[1, 2, 3]$, subarrays are: $[1]$, $[2]$, $[3]$, $[1, 2]$, $[1, 2, 3]$, $[2, 3]$

- $[1] \rightarrow 1$
- $[2] \rightarrow 2$
- $[3] \rightarrow 3$
- $[1, 2] \rightarrow 1+2=3$
- $[2, 3] \rightarrow 2+3=5$
- $[1, 2, 3] \rightarrow 1+2+3=6$

$$1+2+3+3+5+6=20$$

1 appears in subarrays: $[1]$, $[1, 2]$, $[1, 2, 3]$

Contribution of 1 = $1 * 3 = 3$

2 appears in subarrays: $[2]$, $[1, 2]$, $[2, 3]$, $[1, 2, 3]$

Contribution of 2 = $2 * 4 = 8$

3 appears in subarrays: $[3]$, $[2, 3]$, $[1, 2, 3]$

Contribution of 3 = $3 * 3 = 9$

Hence sum of all subarrays can be said as: $3 + 8 + 9 = 20$.

```
n = arr.length
long totalsum = 0
for i from 0 to n - 1
    long res = arr[i] * (i + 1) * (n - i)
    totalsum = totalsum + res
end for
print totalsum
```

- $(i + 1) \rightarrow$ Number of starting indices from which a subarray that includes $\text{arr}[i]$ can begin.
- $(n - i) \rightarrow$ Number of ending indices at which a subarray that includes $\text{arr}[i]$ can end.

Let's take a small array:

$\text{arr} = [3, 1, 2]$

$n = 3$

All possible subarrays are:

[3]

[3, 1]

[3, 1, 2]

[1]

[1, 2]

[2]

And their sums:

$3 + 4 + 6 + 1 + 3 + 2 = 19$

So total sum = 19

Minimum number of partitions.

Given a string `s`
find the minimum number of partitions of a string `s` such that each partition is a word from the dictionary,

You only want the minimum number of valid words to cover the entire string.

```
s = "pineapplepenapple"
dict = ["pine", "apple", "pen", "applepen", "pineapple"]
Partitions:
```

"pine" + "apple" + "pen" + "apple" → 4 partitions

"pineapple" + "pen" + "apple" → 3 partitions (minimum)

"pine" + "applepen" + "apple" → 3 partitions (minimum)

Minimum partitions = 3, but some partitions have 4 parts.

```
int[] dp = new int[len + 1]
Arrays.fill(dp, Integer.MAX_VALUE)
dp[0] = 0
for (int i = 1; i <= len; i++) {
    for (int j = 0; j < i; j++) {
        String sub = s.substring(j, i)
        if (dp[j] != Integer.MAX_VALUE && dict.contains(sub))
            dp[i] = Math.min(dp[i], dp[j] + 1)
    }
}
if (dp[len] == Integer.MAX_VALUE)
    System.out.println(-1)
else
    System.out.println(dp[len])
```

- len = 13 (length of "applepenapple")

We fill `dp[]` from `i=1` to 13.

i = 1:

Check substrings `s[j..i-1]`:

- j=0: `s[0..0]` = "a" → not in dict → no update → `dp[1]` = ∞

i = 2:

- j=0: "ap" → no
 - j=1: "p" → no
- `dp[2]` = ∞

i = 3:

- j=0: "app" → no
 - j=1: "pp" → no
 - j=2: "p" → no
- `dp[3]` = ∞

i = 4:

- j=0: "appl" → no
- j=1: "ppl" → no

- $j=2$: "pl" → no
- $j=3$: "l" → no
- $dp[4] = \infty$

i = 5:

- $j=0$: "apple" → yes in dict and $dp[0] = 0$
→ $dp[5] = \min(\infty, 0 + 1) = 1$
- other j 's ignored since $dp[j]$ is ∞

i = 6:

- $j=0$: "applep" → no
- $j=1..5$ substrings no match
- $dp[6] = \infty$

i = 7:

- $j=0..6$ no substring in dict
- $dp[7] = \infty$

i = 8:

- $j=0..7$ no substring in dict
- $dp[8] = \infty$

i = 9:

- $j=0..4$ no
- $j=5$: "pen" → yes in dict and $dp[5] = 1$
→ $dp[9] = \min(\infty, 1 + 1) = 2$

i = 10..12:

No valid substrings in dict

$dp[10] = \infty, dp[11] = \infty, dp[12] = \infty$

i = 13:

- $j=0..8$ no
- $j=9$: "apple" → yes and $dp[9] = 2$
→ $dp[13] = \min(\infty, 2 + 1) = 3$

- $dp[5] = 1$ means minimum 1 partition for "apple"
- $dp[9] = 2$ means minimum 2 partitions for "applepen"
- $dp[13] = 3$ means minimum 3 partitions for full string

So the minimum partitions to segment "applepenapple" into dictionary words = 3

Longest consecutive subsequence

03 June 2025 14:43

Given an array of n integers, find the **length of the longest subsequence** of consecutive integers (they can appear in any order).

- Duplicates and order don't matter.
- Subsequence (not necessarily contiguous in the array).

Input: arr = [2, 6, 1, 9, 4, 5, 3]

Output: 6

Explanation: The subsequence is [1, 2, 3, 4, 5, 6]

Approach 1: Brute Force

1. Initialize variables

- max_len = 1 → keeps track of longest subsequence length

2. Outer loop (i = 0 to n-1)

Start with each element arr[i] and assume it to be the **starting point** of a potential sequence.

- Let x = arr[i]
- Initialize count = 1 to count this as the first element in sequence

3. Inner loop (j = 0 to n-1)

Search for x+1 in the array:

- If arr[j] == x+1, it means next number in sequence is found
- Then:
 - Increment count
 - Update x = x + 1
 - Reset j = -1 to **restart** search for the next x+1

4. Update the maximum length

After the inner loop ends:

- If count > max_len, update max_len = count

5. Repeat outer loop for all i

6. Return max_len

Time: $O(n^2)$

Space: $O(1)$

approach 2 using sorting

• Sort the array

- So that consecutive numbers are adjacent.

• Initialize variables

- curr_len = 1 → Current sequence length
- max_len = 1 → Max sequence length

• Loop from index 1 to n-1

- If arr[i] == arr[i-1] + 1 → consecutive → increment curr_len
- If arr[i] == arr[i-1] → duplicate → skip (no change)
- Else → not consecutive → reset curr_len = 1

• Update max length after each step

- max_len = max(max_len, curr_len)

• Return max_len

approach 3: optimized using hashset

- For each number, check **only if it's the start** of a sequence (i.e., num - 1 is **not** in the set).
- Then, expand the sequence forward (num + 1, num + 2, ...) until it's broken.

• Add all elements to a HashSet

• Initialize max_len = 0

• Loop over each number num in the array:

- If num - 1 is **not** in the set → num is the **start of a new sequence**

- Set `current = num`
- Count how many `current + 1`, `current + 2`, etc. are in the set using loop
- Track length of this sequence using count (`count++`, `current++`)
- Update `max_len` if `count > max_len` to `count`
- Return `max_len`

Input: `arr = [2, 6, 1, 9, 4, 5, 3]`

`i = 0, arr[i] = 2`

- Is `2 - 1 = 1` in the set? → YES
- → So 2 is NOT the start of a sequence → skip

`i = 1, arr[i] = 6`

- Is `6 - 1 = 5` in the set? → YES
- → Not a start → skip

`i = 2, arr[i] = 1`

- Is `1 - 1 = 0` in the set? → NO
- → So 1 is the start of a new sequence

`current = 1`

`count = 1`

Loop and check for `current + 1` while it's in the set:

Step current Exists in set count

1	2	yes	2
2	3	yes	3
3	4	yes	4
4	5	yes	5
5	6	yes	6
6	7	NO	stop

So, this sequence: `[1, 2, 3, 4, 5, 6]`

`max_len = max(0, 6) = 6`

Approach	Time	Space	Remarks
Brute	$O(n^2)$	$O(1)$	
Sorting	$O(n \log n)$	$O(1)$	
HashSet	$O(n)$	$O(n)$	

Maximum Subarray Sum (Kadane's Algorithm)

05 November 2024 01:25

Given an array of integers (can be positive, negative, or zero), find the contiguous subarray which has the maximum sum, and return that sum.

Input: `arr[] = {2, 3, -8, 7, -1, 2, 3}`

Output: 11

Explanation: The subarray `{7, -1, 2, 3}` has the largest sum 11.

Naive Approach

- Initialize `res = -∞` (or `Integer.MIN_VALUE` in Java)
- Loop `i` from `0` to `n - 1`:
 - Initialize `sum = 0`
 - Loop `j` from `i` to `n - 1`:
 - Add `arr[j]` to `sum`
 - Update `res = max(res, sum)`
- After both loops, `res` will contain the maximum subarray sum

Optimal Approach: Kadane's Algorithm

The idea of Kadane's algorithm is to traverse over the array from left to right and for each element, find the maximum sum among all subarrays ending at that element. The result will be the maximum of all these values.

While traversing, keep a running `curr_sum`.

- If `curr_sum` becomes negative, reset it to `0`.
- Track the max seen so far using `max_sum`.

```
n = scanner.nextInt()
```

```
for i from 0 to n - 1
    arr[i] = scanner.nextInt()
end for
```

```
res = arr[0]
temp = arr[0]
```

```
for i from 1 to n - 1
    temp = max(arr[i], temp + arr[i])
    res = max(res, temp)
```

output `res`

Input: `{2, 3, -8, 7, -1, 2, 3}`

i	arr[i]	curr_sum	max_sum
0	2	2	2
1	3	5	5
2	-8	-3	5
3	7	7	7
4	-1	6	7
5	2	8	8
6	3	11	11

Final Answer = 11

Index: 0 1 2 3 4 5 6
 Array: [2, 3, -8, 7, -1, 2, 3]
 ^^^^^^^^^^^^^^^

Max Subarray → Sum = 11

Time: $O(n)$
 Space: $O(1)$

subarray with 0 sum

03 June 2025 13:01

Given an array of **positive and negative integers**, check if **there exists a subarray** (of size ≥ 1) **whose sum is 0**.

Sample Inputs:

1. {4, 2, -3, 1, 6} $\rightarrow$ true $\rightarrow$ subarray [2, -3, 1]
2. {4, 2, 0, 1, 6} $\rightarrow$ true $\rightarrow$ element 0 itself
3. {-3, 2, 3, 1, 6} $\rightarrow$ false

Approach 1: Brute Force (Naive)

Check **sum of all subarrays**. If any subarray sum = 0, return true.

```
boolean found = false;
for(int i = 0; i < n; i++)
    int sum = 0
    for(int j = i; j < n; j++)
        sum += a[j]
        if(sum == 0) {
            found = true;
            break;
        }

    if(found)
        break

print found
```

Time: $O(n^2)$ **Space:** $O(1)$

Prefix Sum + HashSet (Optimal)

A prefix sum, also known as a cumulative sum, is an array where each element represents the sum of all previous elements in the original array, up to and including that element. It's a powerful technique for efficiently calculating the sum of subarrays within an array.

If at any point the **prefix sum repeats**, or is 0, then a subarray with 0 sum exists.

Reason:

If $\text{prefix_sum}[j] == \text{prefix_sum}[i]$, then subarray from (i+1 to j) has sum 0.

{4, 2, -3, 1, 6}

i	a[i]	prefixsum	seen	prefix already seen
0	4	4	{4}	No
1	2	6	{4, 6}	No
2	-3	3	{4, 6, 3}	No
3	1	4	4 already seen	Yes $\rightarrow$ return true

So subarray [2, -3, 1] has sum 0.

Time: $O(n)$

Space: $O(n)$

```
HashSet<Integer> seen = new HashSet<>()
    int prefix = 0
    boolean found = false
    for(int i = 0; i < n; i++)
        prefix += a[i]
        if(prefix == 0 || seen.contains(prefix)) {
            found = true
            break
        }
        seen.add(prefix)

print(found)
```

binary search with first and last occurrence

24 May 2025 14:26

Given an array of integers `nums` sorted in increasing order, find the starting and ending position of a given target value.

If target is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`

Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`

Output: `[-1,-1]`

```
while (low <= high)
{
    mid = (low + high) / 2
    if (a[mid] == key)
    {
        res = mid
        high = mid - 1
    }
    else if (a[mid] < key)
        low = mid + 1
    else
        high = mid - 1
}
return [res, -1]
```

Peak Element

02 June 2025 12:58

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 2

Explanation: 3 is a peak element and your function should return the index number 2.

Example 2: `0 1 2 3 4`

Input: `nums = [1,2,1,3,5,6,4]`

Output: 5

Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

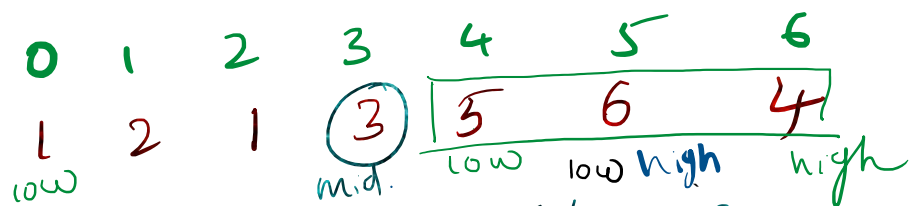
```
int low = 0;
int high = n - 1;
```

```
while (low < high) {
    int mid = (low + high) / 2;

    // Check if mid is greater than its next element
    if (nums[mid] > nums[mid + 1])
        high = mid; // Continue searching in the left half
    else
        low = mid + 1; // Continue searching in the right half
}
```

```
return low;
```

Since $low == high$
return low



① $3 > 5 \times$
 $low = mid + 1 = 4$

② $4 + 6 / 2 = 5$
 $6 > 4 \checkmark$
 $high = mid = 5$

③ $4 + 5 / 2 = 4$
 $5 > 6 \times$ $low = mid + 1 = 5$

In binary search for a value:

- If `a[mid] > target`, you do: `high = mid - 1`
- Because you know `mid` is not the answer

But in **peak search**:

- If `a[mid] > a[mid + 1]`, it means you are in the **decreasing slope** of a bitonic/peak region
 - So the **peak might be at mid itself**, or somewhere to the left
 - Thus, you do: `high = mid` (not `mid - 1`)

Pivot Element

02 June 2025 12:58

Pivot element is the only element in input array which is smaller than it's previous element.

A pivot element divides a sorted rotated array into two monotonically increasing array.

Find such pivot element using recursive binary search.

input=7

4 5 6 7 1 2 3

output=Pivot element is: 1

*/

```
int pivot = findPivot(arr, 0, n - 1)
System.out.println("Pivot element is: " + pivot)
```

```
int findPivot(int[] arr, int low, int high)
{
    if (low > high)
        return arr[0] // Array not rotated, return first element

    if (high == low)
        return arr[low] // Only one element left

    int mid = (low + high) / 2

    // Check if mid+1 is the pivot
    if (mid < high && arr[mid] > arr[mid + 1])
        return arr[mid + 1]

    // Check if mid is the pivot
    if (mid > low && arr[mid] < arr[mid - 1])
        return arr[mid]

    if (arr[low] < arr[mid])
        return findPivot(arr, mid + 1, high) // Search in the right half
    else
        return findPivot(arr, low, mid - 1) // Search in the left half
}
```

Handwritten notes illustrating the pivot search process for the array [4, 5, 6, 7, 1, 2, 3]:

Indices: 0 1 2 3 4 5 6
Values: 4 5 6 7 1 2 3

Calculation: $(0 + 6) / 2 = 3$

Comparison: $7 > 1$ (marked with X), $7 < 6$ (marked with X)

Conclusion: $arr[0] < arr[3]$
 $4 < 7$ (marked with checkmark)

Next step: $\text{findPivot}(arr, 4, 6)$

arr[low] < arr[mid]

- The subarray from low to mid is **sorted** (i.e., increasing).
- So the **pivot can't lie in [low..mid]**, because it's fully sorted.
- Hence, search in the **right half**: mid + 1 to high.

```
int[] arr = {1};
int low = 0, high = 0;
int mid = (0 + 0) / 2 = 0;
Now arr[mid - 1] → arr[-1] → ArrayIndexOutOfBoundsException
```

So, to safely check arr[mid - 1], we must ensure: mid > low

Prime number problems

04 June 2025 14:45

Print prime numbers upto given N

Print prime numbers upto given count

Print prime numbers from given range

sieve of eratosthenesis

24 May 2025 14:26

```
int n = scanner.nextInt()

boolean[] prime = new boolean[n + 1]

Arrays.fill(prime, true)

for (int p = 2; p * p <= n; p++)
    if (prime[p])
        for (int i = p * p; i <= n; i += p)
            prime[i] = false

for (int p = 2; p <= n; p++)
    if (prime[p])
        System.out.print(p + " ")
```

// Time Complexity: $O(n \log \log n)$
// Space Complexity: $O(n)$

```
prime[] = [F, F, T, T, T, T, T, T, T, T, T]
Index      0  1  2  3  4  5  6  7  8  9 10
p = 2: mark 4, 6, 8, 10 as false
```

p = 3: mark 9 as false

Final:

```
prime[] = [F, F, T, T, F, T, F, T, F, F, F]
```

Primes:

2	3	5	7
---	---	---	---

Pivot Index Prefix Sum

02 June 2025 13:32

Input: nums = [1, 7, 3, 6, 5, 6]

Expected Output: 3

Naive Approach

For each index i , calculate:

- left_sum = sum from index 0 to $i-1$
- right_sum = sum from index $i+1$ to $n-1$

```
for (int i = 0; i < n; i++) {
    int left_sum = 0;
    for (int j = 0; j < i; j++)
        left_sum += nums[j];

    int right_sum = 0;
    for (int j = i + 1; j < n; j++)
        right_sum += nums[j];

    if (left_sum == right_sum)
        return i;
}
return -1
```

The **efficient algorithm** avoids recomputing left and right sums every time by:

- Precomputing the **total sum**.
- Tracking the **left sum** while iterating.
- Then computing the **right sum** using:
 $\text{right_sum} = \text{total} - \text{left_sum} - \text{nums}[i]$

because:

$$\text{left_sum} + \text{nums}[i] + \text{right_sum} = \text{total}$$

$$\rightarrow \text{right_sum} = \text{total} - \text{left_sum} - \text{nums}[i]$$

So, instead of recomputing left and right sums for every index, we use just a running left sum and a constant-time expression for right sum.

Optimized Approach

```
int total = 0
for(int i = 0; i < n; i++)
    total += nums[i]

int left_sum = 0
for(int i = 0; i < n; i++)
```



```

        int right_sum = total - left_sum - nums[i]
        if(left_sum == right_sum) {
            print(i)
            return
        }
        left_sum += nums[i]
    }
    print(-1)
}

```

Input: nums = [1, 7, 3, 6, 5, 6]

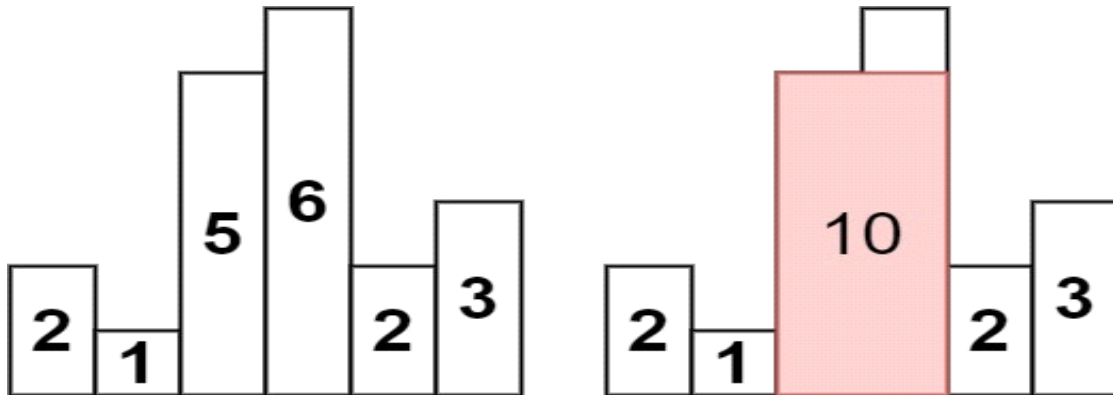
Expected Output: 3

i	nums[i]	left_sum	right_sum = total - left_sum - nums[i]	left_sum == right_sum
0	1	0 left_sum=0+1=1	28 - 0 - 1 = 27	no
1	7	1 left_sum = 1 + 7 = 8	28 - 1 - 7 = 20	no
2	3	8 left_sum = 8 + 3 = 11	28 - 8 - 3 = 17	no
3	6	11	28 - 11 - 6 = 11	YES, return index 3

Largest Rectangle in Histogram

03 June 2025 10:11

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return *the area of the largest rectangle in the histogram*.



Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

1. Naïve Approach - Brute Force

Time $O(n^2)$

Space $O(1)$

For each bar:

- Expand **left** and **right** as far as the bars are **greater than or equal** to the current bar's height.
- Compute area = height of current bar $\times$ width
- Keep track of the **maximum** area found

If you are at index i , and heights[i] = 5,
then you can include as many consecutive bars on the **left and right** that are ≥ 5 .

Index: 0 1 2 3 4 5
heights = [2, 1, 5, 6, 2, 3]
Let's choose $i = 2$, so:

heights[2] = 5

Step 1: Expand Left

Start at $i = 2$, move left:

$i = 1 \rightarrow \text{heights}[1] = 1 < 5 \rightarrow \text{STOP}$

So left boundary = 2

Step 2: Expand Right

Start at $i = 2$, move right:

$i = 3 \rightarrow \text{heights}[3] = 6 \geq 5 \rightarrow \text{OK}$

$i = 4 \rightarrow \text{heights}[4] = 2 < 5 \rightarrow \text{STOP}$

So right boundary = 3

Rectangle formed:

Bars: [5, 6]

Indices: 2 3

Height: 5

Width: $2 \rightarrow (3 - 2 + 1)$

Area: $5 \times 2 = 10$

Optimized Approach

- for **each bar**, we want to find:
 - The **first smaller bar on the left** $\rightarrow$ tells us how far we can go left
 - The **first smaller bar on the right** $\rightarrow$ tells us how far we can go right
- Using these two, we compute:

$\text{width} = \text{right_index} - \text{left_index} - 1$

$\text{area} = \text{height} * \text{width}$

- To do this efficiently in $O(n)$ time, we can use stack.
-

Stack maintains indices of increasing heights.

Step 1: Initialization

- Let stack be an empty stack storing **indices** of bars.
- Let $\text{max_area} = 0$

Step 2: Iterate over the heights

For each i from 0 to $n-1$:

1. While the **stack is not empty** and $\text{heights}[i] < \text{heights}[\text{stack.peek()}]$:
 - It means the current bar breaks the increasing trend.
 - Pop the top index from the stack $\rightarrow \text{top} = \text{stack.pop}()$
 - Compute $\text{height} = \text{heights}[\text{top}]$
 - Compute width:
 - If stack is empty: $\text{width} = i$
 - Else: $\text{width} = i - \text{stack.peek()} - 1$
 - Compute $\text{area} = \text{height} \times \text{width}$
 - Update max_area
2. Push current index i into the stack.

Step 3: Clean up remaining stack

After reaching end of the array ($i == n$), some bars might still be in the stack.

While stack is not empty:

- Pop $\text{top} = \text{stack.pop}()$
- Compute $\text{height} = \text{heights}[\text{top}]$
- Compute width:

- If stack is empty: width = n
- Else: width = n - stack.peek() - 1
- Compute area = height × width
- Update max_area

example test case:

heights = [2,1,5,6,2,3].

Stack = index positions

i = 0

- h[0] = 2 → stack is empty → push 0
stack = [0]

i = 1

- h[1] = 1 < h[stack.peek()] → pop
→ top = 0, h[0] = 2, width = 1 since stack is empty
→ area = 2×1 = 2
max_area = 2, push 1
stack = [1]

i = 2

- h[2] = 5 ≥ h[1] → push
stack = [1, 2]

i = 3

- h[3] = 6 ≥ h[2] → push
stack = [1, 2, 3]

i = 4

- h[4] = 2 < h[3]
→ pop 3 → height = 6, width = 1(4-2-1) → area = 6
→ pop 2 → height = 5, width = 2(4-1-1) → area = 10
max_area = 10, push 4
stack = [1, 4]

and so on.

Time Complexity = O(n)

- n pushes + n pops = O(n)
- Everything else inside the loop is constant time.

Kth Smallest element using Priority queue (max heap)

04 June 2025 15:29

Given an $n \times n$ matrix where each of the rows and columns is sorted in **ascending** order, and an integer k , return the **kth smallest** element in the matrix.

Sample input=

```
1 5 9
10 11 13
12 13 15
k = 8
```

Sample output=

13

Naive approach

Idea:

Flatten the matrix into a 1D array, sort it, and return the k -th element.

● **Time Complexity:**

- Flattening: $O(n^2)$
- Sorting: $O(n \log n)$
- Overall: $O(n^2 \log n)$

● **Space Complexity:** $O(n^2)$

Better Approach (Using max Heap)

This property states that for every node in the heap, its value is greater than or equal to the values of its children. Consequently, the largest element in the entire heap is always located at the root.

- Traverse the entire matrix ($n \times n$ elements).
- For each element:
 - Push into a **max heap**
 - If heap size exceeds k , **remove the top (maximum) element**.
- After traversing all n^2 elements:
 - Heap contains **exactly the k smallest elements** from the matrix.
 - **Top of heap = k -th smallest**

Given matrix:

```
10 20 30 40
15 25 35 45
24 29 37 48
32 33 39 50
```

And $k = 3$

We want to find the **3rd smallest element**.

output=20

Heap state after each insertion:

Step Inserted Heap (top is largest) Explanation

1	10	[10]	size $\leq k$
---	----	------	---------------

2	20	[20, 10]	size $\leq k$
3	30	[30, 10, 20]	size = k
4	40	[30, 10, 20]	remove 40 $\rightarrow$ not inserted
5	15	[20, 10, 15]	remove 30 $\rightarrow 30 > 15$
6	25	[20, 10, 15]	remove 25 $\rightarrow 25 > 20$
7	35	[20, 10, 15]	remove 35 $\rightarrow 35 > 20$
8	45	[20, 10, 15]	remove 45 $\rightarrow 45 > 20$
9	24	[20, 10, 15]	remove 24 $\rightarrow 24 > 20$
10	29	[20, 10, 15]	remove 29 $\rightarrow 29 > 20$
11	37	[20, 10, 15]	remove 37 $\rightarrow 37 > 20$
12	48	[20, 10, 15]	remove 48 $\rightarrow 48 > 20$
13	32	[20, 10, 15]	remove 32 $\rightarrow 32 > 20$
14	33	[20, 10, 15]	remove 33 $\rightarrow 33 > 20$
15	39	[20, 10, 15]	remove 39 $\rightarrow 39 > 20$
16	50	[20, 10, 15]	remove 50 $\rightarrow 50 > 20$

Final Heap:

[20, 10, 15]

- Top of heap (largest among k smallest) = **20**
- So, the **3rd smallest element is 20**

visualize heap as a **binary tree (max heap)** at each main step:

After inserting 10, 20, 30:

```

      30
     /  \
    10   20

```

Insert 40:

```

      30
     /  \
    10   20

```

$\rightarrow$ remove 40 $\rightarrow$ ignored

Insert 15:

```

      20
     /  \
    10   15

```

$\rightarrow$ 30 is removed to keep size = k

Heap always keeps the **k smallest**, and throws away anything bigger than current top.

```
PriorityQueue<Integer> heap = new PriorityQueue<>(Collections.reverseOrder())
```

```

for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        heap.offer(mat[i][j])
        if(heap.size() > k)
            heap.poll()
    }
System.out.println(heap.peek())

```

Each **insert** and **remove operation** on the heap costs **$O(\log k)$**

Time: $O(n^2 \times \log k)$

Space = $O(n^2 + k)$

aggressive cows

24 May 2025 14:26

You are given an array with unique elements of stalls[], which denote the position of a stall. You are also given an integer k which denotes the number of aggressive cows. Your task is to assign stalls to k cows such that the minimum distance between any two of them is the maximum possible.

Examples :

Input: stalls[] = [1, 2, 4, 8, 9], k = 3

Output: 3

Explanation: The first cow can be placed at stalls[0],

the second cow can be placed at stalls[2] and

the third cow can be placed at stalls[3].

The minimum distance between cows, in this case, is 3, which also is the largest among all possible ways.

Input: stalls[] = [10, 1, 2, 7, 5], k = 3

Output: 4

Explanation: The first cow can be placed at stalls[0],

the second cow can be placed at stalls[1] and

the third cow can be placed at stalls[4].

The minimum distance between cows, in this case, is 4, which also is the largest among all possible ways.

Input: stalls[] = [2, 12, 11, 3, 26, 7], k = 5

Output: 1

Explanation: Each cow can be placed in any of the stalls, as the no. of stalls are exactly equal to the number of cows.

The minimum distance between cows, in this case, is 1, which also is the largest among all possible ways.

Naive Approach

- try all possible minimum distances d from 1 up to the maximum possible distance (max(stalls) - min(stalls)).
- For each distance d, try to place all cows in stalls so that the distance between any two cows is at least d.
- The largest d for which placement is possible is the answer.

Time:

$O(\text{max_dist} * n)$

(since max_dist ranges from 1 to (max-min), and for each, we do a linear scan)

stalls = [1, 2, 4, 8, 9]

k = 3 (number of cows)

Sort first step: {1,2,4,8,9}

max_distance = stalls[max] - stalls[min] = 9 - 1 = 8

Step 2: Try each distance from 1 to 8 (inclusive)

Distance d	Is it possible to place 3 cows with at least d?	How cows are placed?	Result
1	Yes	Cows at [1, 2, 4]	Possible
2	Yes	Cows at [1, 4, 8]	Possible
3	Yes	Cows at [1, 4, 8]	Possible
4	Yes	Cows at [1, 4, 8]	4-1=3 not Possible
5	No	Try [1, 8, ?] (9-8=1 < 5 no)	Not Possible
6	No	Try [1, 8, ?] (9-8=1 < 6 no)	Not Possible
7	No	Try [1, 8, ?] (9-8=1 < 7 no)	Not Possible
8	No	Only 1 cow can be placed at 1	Not Possible

Approach 2 (Optimal - Binary Search on Answer):

1. Sort the stall positions.

2. Binary Search:

 o low = 1

 o high = max_distance = last - first

3. For a mid value (distance), check if it's possible to place k cows such that minimum distance between any two is at least mid.

4. If yes → try bigger distance.

 If no → try smaller.


```

Arrays.sort(stalls)

int low = 1;
int high = stalls[n - 1] - stalls[0]
int ans = 0

while (low <= high) {
    int mid = (low + high) / 2

    if (is_possible(stalls, n, k, mid))
        ans = mid;
        low = mid + 1;
    else
        high = mid - 1;
}

print(ans)
}

boolean is_possible(int[] stalls, int n, int cows, int dist) {
    int count = 1
    int last_pos = stalls[0]

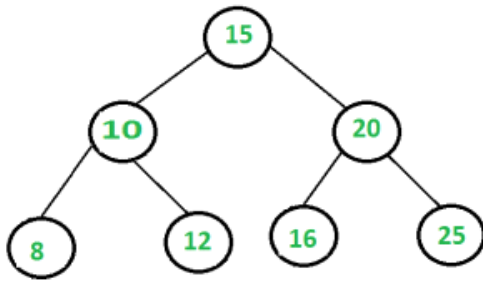
    for (int i = 1; i < n; i++)
        if (stalls[i] - last_pos >= dist) {
            count++
            last_pos = stalls[i]
        }

    return count >= cows
}

```

sum of all nodes in Binary Tree

04 June 2025 15:17



Output=106

```
int compute_sum(Node root) {  
    if (root == null) return 0;  
    return root.data + compute_sum(root.left) + compute_sum(root.right)  
}
```

if tree is skewed stack approach is better

```
Stack<Node> stack = new Stack<>()  
stack.push(root)  
while (!stack.isEmpty()) {  
    Node node = stack.pop()  
    sum += node.data  
    if (node.right != null) stack.push(node.right)  
    if (node.left != null) stack.push(node.left)  
}
```

BFS and DFS traversal

04 June 2025 15:34

